



REVIEW

Prompt Injection Attacks on Large Language Models: A Survey of Attack Methods, Root Causes, and Defense Strategies

Tongcheng Geng^{1,#}, Zhiyuan Xu^{2,#}, Yubin Qu^{3,*} and W. Eric Wong⁴

¹Department of Information and Network Security, The State Information Center, Beijing, 100032, China

²Department of Mechanical Engineering, Hohai University, Changzhou, 213200, China

³School of Information Engineering, Jiangsu College of Engineering and Technology, Nantong, 226001, China

⁴Department of Computer Science, University of Texas at Dallas, Dallas, TX 75080, USA

*Corresponding Author: Yubin Qu. Email: quyubin@hotmail.com

#Tongcheng Geng and Zhiyuan Xu contributed equally to this work

Received: 01 October 2025; Accepted: 25 November 2025; Published: 10 February 2026

ABSTRACT: Large language models (LLMs) have revolutionized AI applications across diverse domains. However, their widespread deployment has introduced critical security vulnerabilities, particularly prompt injection attacks that manipulate model behavior through malicious instructions. Following Kitchenham's guidelines, this systematic review synthesizes 128 peer-reviewed studies from 2022 to 2025 to provide a unified understanding of this rapidly evolving threat landscape. Our findings reveal a swift progression from simple direct injections to sophisticated multimodal attacks, achieving over 90% success rates against unprotected systems. In response, defense mechanisms show varying effectiveness: input preprocessing achieves 60%–80% detection rates and advanced architectural defenses demonstrate up to 95% protection against known patterns, though significant gaps persist against novel attack vectors. We identified 37 distinct defense approaches across three categories, but standardized evaluation frameworks remain limited. Our analysis attributes these vulnerabilities to fundamental LLM architectural limitations, such as the inability to distinguish instructions from data and attention mechanism vulnerabilities. This highlights critical research directions such as formal verification methods, standardized evaluation protocols, and architectural innovations for inherently secure LLM designs.

KEYWORDS: Prompt injection attacks; large language models; defense mechanisms; security evaluation

1 Introduction

The escalating scale and complexity of large language models have led to a sharp increase in security threats. LLM security vulnerabilities can trigger systemic failures, causing significant losses and widespread impact. For instance, in 2023, a vulnerability was discovered in a ChatGPT plugin named “Chat with Code” where a prompt injection payload on a webpage could modify GitHub repository permission settings, turning private repositories public [1–4]. Another case in May 2024 arose from OpenAI's introduction of long-term memory functionality; research found that prompt injection attacks could embed malicious instructions into ChatGPT's memory, creating spyware that continuously and covertly steals all user chat conversations [4,5]. Furthermore, studies have demonstrated how a simple prompt injection payload on a webpage can trick Claude Computer Use into downloading and running malicious software, turning the user's computer into part of a botnet [5,6]. The significant losses caused by LLM security vulnerabilities underscore the need for robust security protection. Defense against prompt injection attacks, in particular,



plays a crucial role throughout the entire AI system lifecycle. Prompt injection attack research aims to identify and defend against malicious input manipulation targeting large language models.

Historically, prompt injection attacks have evolved from simple manual crafting to sophisticated automated generation, marking a clear progression in methodology. The earliest prompt injection attacks emerged in 2022 when Perez et al. [7] first systematically introduced direct injection attacks using simple commands like “ignore previous instructions” to override model instructions, with early techniques primarily relying on explicit instruction overriding and role-playing methods [8]. The year 2023 witnessed the rise of indirect injection attacks, where attackers began leveraging external data sources (webpages, documents, emails) as attack vectors through Retrieval-Augmented Generation (RAG) systems [8,9], representing a strategic shift from confrontational to more subtle indirect manipulation. Entering 2024, prompt injection technology advanced into the multimodal era with the proliferation of models like GPT-4V and Claude 3. This enabled attackers to explore injection possibilities through non-text modalities such as images and audio [10,11]. Visual prompt injection attacks, for example, have successfully bypassed traditional text filtering by embedding imperceptible malicious instructions in images [12–14]. Correspondingly, defense strategies have evolved from passive input preprocessing and rule-based filtering [15,16] to comprehensive multi-layered approaches that incorporate system architecture-level protections, model-level security enhancements through adversarial training [17,18], and integrated defense-in-depth systems [19,20].

Currently, four surveys exist on prompt injection attacks [21–24], each contributing different perspectives to the field. Peng et al. [21] systematically review LLM security issues, including accuracy, bias, content detection, and adversarial attacks, while Rababah et al. [22] provide the first systematic knowledge framework explicitly classifying prompt attacks into jailbreaking, leaking, and injection categories with a five-category response evaluation framework. Mathew [23] offers a comprehensive analysis of emerging attack techniques such as HOUYI (Names of mythological figures from ancient China), RA-LLM (Robustly Aligned LLM), and StruQ (Structured Queries), evaluating their effectiveness on mainstream models, while Kumar et al. [24] propose a coherent framework organizing attacks based on prompt type, trust boundary violations, and required expertise. However, these surveys exhibit significant limitations. First, many adopt broad coverage strategies treating prompt injection as a subset of LLM security rather than providing in-depth technical analysis. Second, they lack unified classification standards and systematic frameworks that fully capture the diversity of attack techniques and implementation mechanisms. Third, they show deficiencies in systematically organizing defense strategies without establishing clear correspondences between defense approaches and attack types. Furthermore, existing work has inadequately analyzed the dynamic adversarial relationship between attack and defense technologies, failing to reveal co-evolution patterns and future development trends in this rapidly evolving field.

To address existing research gaps, this survey provides a comprehensive analysis framework for prompt injection attacks and defenses with four main contributions:

1. **Systematic Attack Classification System:** We construct a multi-dimensional classification framework covering direct injection, indirect injection, and multimodal injection based on attack vector, target, and technical implementation;
2. **Root Cause Attribution Analysis:** We examine the fundamental causes of successful prompt injection attacks from philosophical, technical, and training perspectives;
3. **Comprehensive Defense Strategy Review:** We systematically organize defense mechanisms across input preprocessing, system architecture, and model levels;

This paper systematically explores prompt injection attacks, their attribution, and defense strategies in large language models. We detail the evolution of attack techniques [Section 1](#), the systematic review

methodology employed [Section 2](#), and classify current attack methods [Section 3](#). Furthermore, we analyze the underlying causes of vulnerability [Section 4](#), categorize defense mechanisms [Section 5](#), and survey evaluation platforms and metrics [Section 6](#), concluding with a summary of findings [Section 7](#).

2 Methodology

In this paper, we conducted a systematic literature review following the guidelines of Kitchenham [25], Zhang et al. [26], and Niu et al. [27] to ensure a fair and reproducible procedure. The process consisted of three steps: planning, execution, and analysis. In the planning phase, we first identified the research questions and objectives aligned with our research goals. Then, in the execution phase, we conducted research based on the identified objects, selection criteria, and snowballing to obtain a diverse collection of literature on prompt injection attacks. Finally, in the analysis phase, our four co-authors analyzed the selected literature and answered the research questions. Our research process is visually presented in [Fig. 1](#).

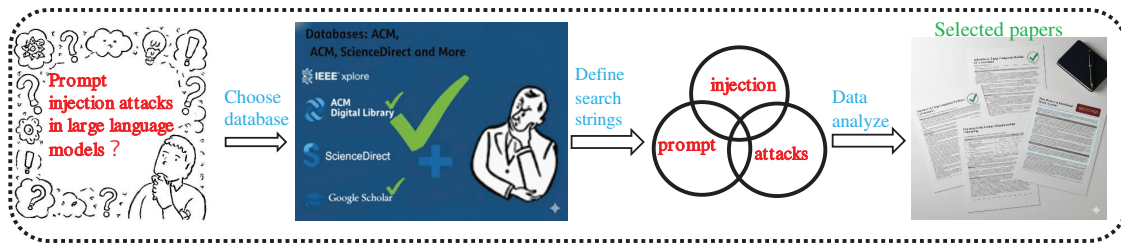


Figure 1: Primary study selection process

2.1 Research Objects

With the rapid growth of large language models, diverse attack techniques are emerging, yet the varied attack methods lead to conceptual ambiguities and overlaps in academic and industry definitions, hindering clear research scopes and comparable results. To address this, we first define prompt injection attacks, clarifying their core mechanisms and distinguishing them from related attacks like adversarial samples, jailbreaks, and data poisoning. This conceptual framework provides clarity and ensures the survey's focus and practicality.

We adopt the framework from Liu et al. [28] to model prompt injection attacks. We represent the backend LLM as a function $f: \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{X}$ is the input space and $\mathcal{Y}$ is the output space. The system involves two tasks: a legitimate task $t = (s_t, x_t)$ with target instruction s_t and target data x_t , and an injected task $e = (s_e, x_e)$ with malicious instruction s_e and injected data x_e . A prompt injection attack is formalized as function $A: \mathcal{X}_t \times \mathcal{S}_e \times \mathcal{X}_e \rightarrow \tilde{\mathcal{X}}$ that generates poisoned input $\tilde{x} = A(x_t, s_e, x_e)$. Let $\mathcal{S} \subseteq \mathcal{Y}$ denote the secure output space, $\mathcal{P}$ denote system prompts, and $\mathcal{U}$ denote user inputs. The attack objective is to construct malicious input $u_{malicious}$ such that $M([p_i, u_{malicious}]) \notin \mathcal{S}$ for some system prompt p_i . The attack succeeds when there exists function A and payload (s_e, x_e) such that:

$$\exists p_i \in \mathcal{P}, \text{ s.t. } f([p_i, A(x_t, s_e, x_e)]) \notin \mathcal{S}$$

and the output similarity to the injected task target exceeds the threshold $\theta \in [0, 1]$, where θ represents the minimum semantic similarity score (typically measured using cosine similarity or other text similarity metrics) required between the model's actual output and the attacker's intended malicious output for the attack to be considered successful. Following Liu et al. [28], we set $\theta = 0.7$ as the default threshold, indicating that an attack is successful when the output achieves at least 70% similarity to the injected task objective.

Jailbreaking attacks bypass safety alignment to generate harmful content, whereas prompt injection attacks manipulate task execution. These attacks exploit language models' instruction-following capabilities, differing in their targets (safety bypass vs. task transformation), scenarios (direct interaction vs. external data pollution), and success conditions (safety bypass vs. task execution). Backdoor attacks embed triggers during training to manipulate model behavior persistently, whereas prompt injection attacks exploit vulnerabilities during inference with dynamic malicious inputs. While both can use natural language as an attack vector, their core differences lie in the attack stage (training vs. inference), trigger conditions, persistence, and detection mechanisms.

2.2 Research Questions

The overall objective of this review is to gain a deeper understanding of the current state of prompt injection attacks and their defense mechanisms, with a particular focus on factors that lead to prompt injection attacks on Large Language Models. To thoroughly understand this topic, this review addresses four research questions. These questions allow us to systematically classify and comprehend current research, identify limitations in prompt injection attack research, and pinpoint future research directions.

1. **What prompt injection attack methods have been proposed?** This identifies and analyzes attack techniques, including implementation mechanisms and payload design.
2. **Why are large language models vulnerable to prompt injection attacks?** This analyzes root causes, including model architecture vulnerabilities, training defects, cognitive limitations, and their interactions.
3. **What defense mechanisms mitigate prompt injection attacks?** This reviews existing strategies, including input-based defenses, model improvements, and system-level protections, analyzing their effectiveness and limitations.
4. **What datasets and evaluation metrics support prompt injection research?** This review research infrastructure included attack datasets, defense evaluation datasets, and metric systems.

2.3 Literature Search Strategy

We first formulated a literature search strategy to search for relevant studies from academic digital libraries effectively. We designed our search strings based on the PICO (Population, Intervention, Comparison, and Outcome) framework [27,29], which is widely used in review and systematic mapping studies [27,30]. The relevant terms for Population, Intervention, Comparison, and Outcome are as follows:

- Population: Large Language Models, LLMs, ChatGPT, GPT, AI systems, conversational AI
- Intervention: prompt injection, jailbreak, adversarial prompts, defense mechanisms, security measures
- Comparison: baseline, comparison, evaluation, benchmark
- Outcome: prompt injection attack, security, robustness, attack success rate, defense effectiveness, vulnerability

Based on the PICO framework, we used the following search string to find relevant articles: ("large language model" OR "LLM" OR "generative AI" OR "ChatGPT" OR "GPT") AND ("prompt injection" OR "prompt injection attack" OR "adversarial prompt" OR "jailbreak" OR "prompt manipulation") AND ("security" OR "attack" OR "prompt injection attack defense") We applied this search string to the nine electronic databases listed in Fig. 1 to search for relevant articles. Since prompt injection attacks are an emerging field, we particularly focused on the arXiv preprint repository and recent conference papers. We searched on 04 August 2025, identifying studies published up to that date. As shown in Fig. 1, we initially retrieved 586 distinct studies: 35 from IEEE Xplore, 18 from ACM Digital Library, four from Science Direct,

nine from Springer Link, six from Wiley InterScience, eight from Elsevier, 243 from Google Scholar, 11 from DBLP, and 252 from ArXiv.

2.4 Literature Selection Criteria

Inclusion/Exclusion Criteria. To identify the articles most relevant to the research questions in our review, we referred to similar studies [24,27,31] and defined our Inclusion Criteria (ICs) and Exclusion Criteria (ECs). Table 1 lists the ICs and ECs. By applying inclusion and exclusion criteria to titles, abstracts, and keywords, we ensured selected studies were English literature published between January 2022 and August 2025 (no time limit for attribution analysis literature), available as peer-reviewed publications or high-quality arXiv preprints with full text access, and specifically addressing prompt injection attacks on large language models following Perez and Ribeiro’s definition [7] of attacks that manipulate LLM behavior through maliciously constructed input prompts. Studies must focus on attack methods, defense mechanisms, or vulnerability assessment while providing empirical evaluation, theoretical analysis, or systematic methodologies. We excluded studies not involving prompt injection targeting LLMs, papers discussing only general AI safety without a specific focus on prompt injection, duplicate studies or reviews lacking novel contributions, documents under four pages or lacking technical details, and studies focusing solely on unrelated adversarial attacks. After applying these criteria, 478 studies were removed, retaining 108 studies in our research pool.

Table 1: Inclusion and exclusion criteria

Inclusion criteria	
IC1	The paper is written in English.
IC2	The paper was published between January 2022 and August 2025.
IC3	The paper is published in a peer-reviewed journal, conference, workshop, or available as a preprint on arXiv, with full-text accessible.
IC4	The paper addresses prompt injection attacks against large language models or related AI systems.
IC5	The paper focuses on attack methods, defense mechanisms, or evaluation of prompt injection vulnerabilities.
IC6	The paper provides empirical evaluation, theoretical analysis, or systematic methodology related to prompt injection.
Exclusion Criteria	
EC1	The paper is not about prompt injection, jailbreaking, or adversarial prompting against LLMs.
EC2	The paper only discusses general AI safety or ethics without a specific focus on prompt injection attacks.
EC3	The paper is a duplicate, survey paper, or does not provide novel contributions to the field.
EC4	The paper is short (less than 4 pages), poster abstract, or lacks sufficient technical detail.
EC5	The paper focuses solely on other types of adversarial attacks (e.g., adversarial examples for computer vision) without relevance to prompt injection.

Snowballing Method. We expanded our initial literature set through a snowballing process, following the guidelines in [32], by iteratively examining references and citations. This method ensured completeness, with forward and backward snowballing ceasing once no new relevant studies emerged, and ultimately adding 20 papers to reach a total of 128 articles.

Quality Assessment. Quality assessment is a crucial step in a review to ensure that we can present the research work appropriately and fairly [25]. We used a quality checklist to evaluate the quality of the studies and excluded those that failed to pass the checklist. Our quality checklist was derived from Hall et al. [33] and modified to suit the characteristics of prompt injection attack research. We mainly assessed original research from four aspects: the originality of technical contributions, the rigor of experimental design, the adequacy of results analysis, and the completeness of ethical considerations, as shown in Table 2. The quality assessment checklist was independently applied to all 128 primary studies by two authors. In case of disagreement, discussions were held to reach a consensus. Ultimately, 128 primary studies were included in the data extraction phase.

Table 2: The quality assessment checklist

Attack method criteria	
AMC1	The paper clearly describes the prompt injection attack methodology or technique.
AMC2	The attack mechanism and underlying principles are well explained.
AMC3	The scope and limitations of the proposed attack are discussed.
Defense mechanism criteria	
DMC1	The defense strategy or mitigation approach is clearly presented (if applicable).
DMC2	The theoretical foundation or rationale behind the defense is explained.
DMC3	The effectiveness and limitations of the defense are analyzed.
Experimental evaluation criteria	
EEC1	The experimental setup and methodology are clearly described.
EEC2	The target models, datasets, or evaluation scenarios are specified.
EEC3	Quantitative results with appropriate metrics are reported.
EEC4	Baseline comparisons or ablation studies are conducted (when applicable).
Reproducibility criteria	
RC1	Implementation details are sufficiently provided for reproduction.
RC2	Code, datasets, or supplementary materials are made available (when possible).

2.5 Data Analysis

Data Extraction. After the initial study selection, we developed a data extraction form (Table 3) to extract data from the primary studies to answer the research questions. As shown in the table, there are a total of 23 fields. The first five rows constitute the metadata of the study, with six fields related explicitly to RQ1 (Attack Methods), six fields related to RQ2 (Vulnerability Attribution Analysis), six fields related to RQ3 (Defense Mechanisms), and the remaining four fields associated with RQ4 (Datasets and Evaluation Metrics). The first author formulated an initial extraction table based on the characteristics of the prompt

injection attack domain, focusing on core elements such as attack techniques, defense strategies, evaluation methods, and research challenges. Then, the two authors conducted a pilot study on ten randomly selected preliminary studies to assess the completeness and usability of the table. During the pilot process, the authors found it necessary to add detailed descriptions to fields such as attack type classification, defense mechanism categories, performance indicators, and statistical analysis to better capture the characteristics of prompt injection attack research. The two authors continuously discussed and refined the table's structure until they reached a consensus. All preliminary studies were distributed between the two authors for independent data extraction from their respective research. The two authors collectively filled the data extraction table using an online form. Finally, the third author checked the extraction table to ensure the correctness of the results and the data extraction consistency.

Table 3: The data extraction form

Field	Input type	Relevant RQ
Paper ID	Auto-fill	metadata
Paper title	Free text	metadata
Publication year	Number	metadata
Publication venue	Free text	metadata
Research focus	Multiple selection	metadata
Attack methodology	Free text	RQ1
Attack type	Multiple selection	RQ1
Target models	Free text	RQ1
Attack success metrics	Multiple selection	RQ1
Evaluation datasets	Free text	RQ1
Experimental setup	Free text	RQ1
Vulnerability factors	Free text	RQ2
Root cause category	Multiple selection	RQ2
Philosophical analysis	Free text	RQ2
Technical limitations	Free text	RQ2
Training deficiencies	Free text	RQ2
Theoretical framework	Free text	RQ2
Defense mechanism	Free text	RQ3
Defense category	Multiple selection	RQ3
Detection approach	Free text	RQ3
Mitigation strategy	Free text	RQ3
Defense effectiveness	Free text	RQ3
Implementation details	Free text	RQ3
Evaluation methodology	Free text	RQ4
Benchmark datasets	Free text	RQ4
Performance metrics	Multiple selection	RQ4
Baseline comparisons	Free text	RQ4
Statistical analysis	Free text	RQ4
Reproducibility	Multiple selection	RQ4

Data Synthesis. The ultimate goal of a survey study is information aggregation to provide an overview of the current state of technology. We extracted quantitative data from the data extraction table to identify and report the results for RQ1, RQ3, and RQ4. For RQ2, we conducted a qualitative analysis to synthesize theoretical analyses regarding the root causes of model vulnerabilities. Specifically, this was to identify reported vulnerability mechanisms, root causes, and gaps in current understanding. During the data extraction process, any discussion explicitly mentioning vulnerability analysis, attack mechanisms, or model limitations in the paper was extracted into the data extraction table. We extracted major themes and manually revised them to categorize vulnerability causes into three levels of issues: architectural, training, and cognitive.

2.6 Literature Search Results

There were 128 articles in the final research pool. The first prompt injection attack method appeared in 2022, which coincides with the widespread application of large language models like ChatGPT. Since 2023, the number of published papers has exploded annually, reflecting the high attention is paid by academia and industry to LLM security issues. This indicates that prompt injection attacks have become a hot research direction in the field of AI security and are still developing rapidly at the time of this study, as shown in Figs. 2 and 3.

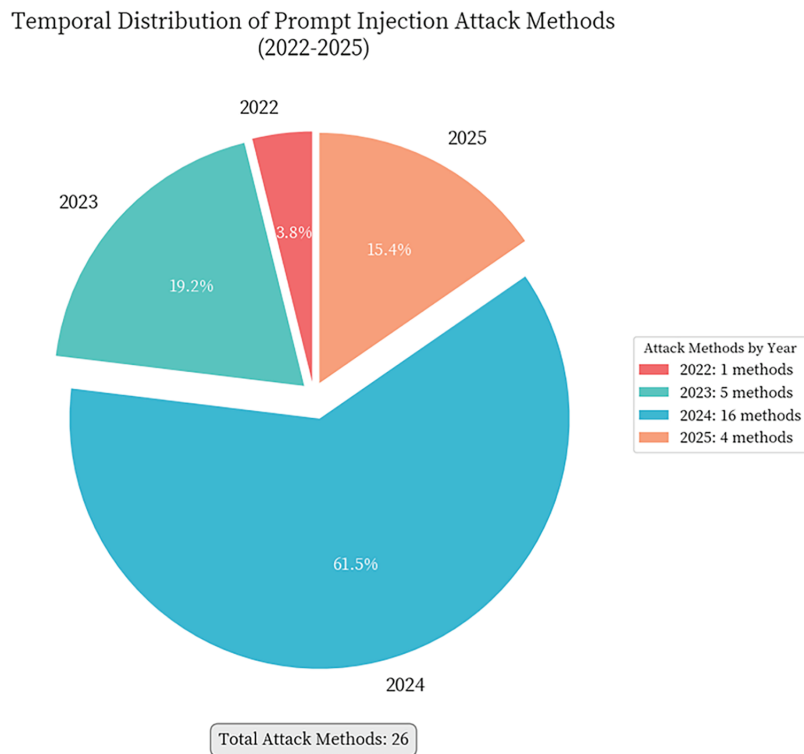


Figure 2: Temporal distribution of prompt injection attack methods (2022–2025). This figure illustrates the rapid evolution of attack techniques, showing the number of novel attack methods proposed each year. The distribution reveals an exponential growth trend, with a significant surge beginning in 2023, coinciding with the widespread deployment of LLM-based applications. The increasing diversity and sophistication of attack methods reflect the expanding attack surface as LLMs are integrated into more complex systems with external tool access, multi-modal capabilities, and agent-based architectures. Each data point represents a distinct attack methodology identified in our systematic literature review

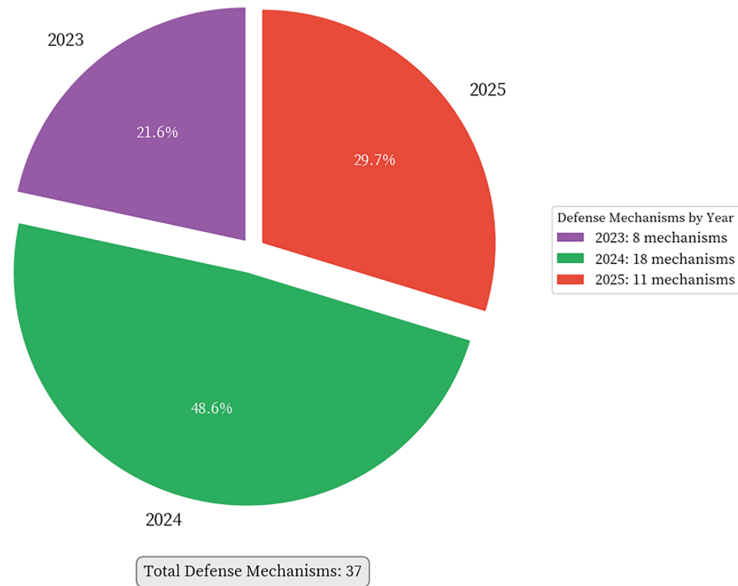
Temporal Distribution of Prompt Injection Defense Mechanisms
(2023-2025)

Figure 3: Temporal distribution of prompt injection defense mechanisms (2023–2025). This figure tracks the development of defense strategies over time, showing the number of novel defense methods proposed each year. The distribution demonstrates a reactive pattern where defense research intensifies following the proliferation of attack methods. The notable increase from 2023 onwards indicates the research community’s growing recognition of prompt injection as a critical security challenge. The temporal lag between attack and defense publications (visible when compared with Fig. 2) highlights the inherent asymmetry in the security arms race, where attackers often maintain a temporal advantage

3 RQ1: What Prompt Injection Attack Methods Have Been Proposed So Far?

We systematically categorized and summarized existing prompt injection attack methods, observing their diversity and evolving trends in carriers, targets, and technical implementations as shown in Table 4. This classification by attack vectors and technical implementations helps understand attack characteristics and informs defense strategies and research directions.

Table 4: Summary of prompt injection attack methods

Index	Year	Authors	Attack vector	Attack target	Core technology
1	2022	Perez and Ribeiro [7]	User input	Goal hijacking	Instruction override attack
2	2023	Greshake et al. [8]	Third-party content	Remote control	Indirect prompt injection
3	2023	Liu et al. [9]	User input	Prompt stealing	HOUYI three-stage attack
4	2023	Shah et al. [34]	Role setting	Safety bypass	Role modulation attack
5	2023	Toyer et al. [35]	Gamified input	Access control	Gamified attack generation

(Continued)

Table 4 (continued)

Index	Year	Authors	Attack vector	Attack target	Core technology
6	2023	Wei et al. [36]	Logic construction	Alignment bypass	Competing objective exploitation
7	2024	Debenedetti et al. [37]	Third-party data	Agent hijacking	Tool invocation attack
8	2024	Hui et al. [38]	Adversarial query	Prompt leakage	PLEak incremental search
9	2024	Kumar et al. [24]	Multi-dimensional Space	Trust breach	Five-layer attack exploitation
10	2024	Kwon et al. [12]	Mathematical function	Safety bypass	Functionalized text injection
11	2024	Lee et al. [39]	Multi-agent	Viral propagation	Prompt infection attack
12	2024	Liu et al. [40]	Gradient optimization	Dynamic target	Momentum gradient search
13	2024	Liu et al. [28]	Mixed input	Task hijacking	Formalized attack framework
14	2024	Pasquini et al. [41]	Learning optimization	Execution trigger	Differentiable search
15	2024	Rehberger [4]	Mixed data	System destruction	Seven system attacks
16	2024	Rossi et al. [42]	Direct/Indirect	System manipulation	Classified injection attack
17	2024	Shi et al. [43]	Candidate response	Judgment manipulation	JudgeDeceiver attack
18	2024	Wang et al. [13]	Multimodal input	Cross-modal hijacking	CrossInject attack
19	2024	Yan et al. [44]	Training data	Virtual execution	Virtual prompt injection
20	2024	Yu et al. [45]	Custom GPT	File leakage	Adversarial generation
21	2024	Zhang et al. [46]	Text input	Goal guidance	KL divergence maximization
22	2024	Zhang et al. [47]	Robot system	Behavior control	LLM robot attack
23	2025	Alizadeh et al. [48]	Execution data	Data leakage	Data flow oriented attack
24	2025	Clusmann et al. [14]	Medical image	Diagnosis manipulation	Vision-language injection
25	2025	Liang et al. [49]	Custom prompt	IP Theft	Perplexity mechanism analysis

(Continued)

Table 4 (continued)

Index	Year	Authors	Attack vector	Attack target	Core technology
26	2025	Wang et al. [50]	Web pixel	Web agent	Environmental prompt injection

3.1 Classification Based on Attack Vectors

3.1.1 Direct Injection Attack Methods

Direct injection attack is the most straightforward form of prompt injection attack, where attackers manipulate the behavior of large language models by directly embedding malicious instructions into user input. This type of attack is characterized by the attack payload being transmitted in the same input channel as the user query, and the attacker attempts to override or bypass the system's original prompt through carefully designed instructions.

Instruction Following Attacks. Instructions following attacks represent a fundamental category of direct injection attacks that exploit the model's tendency to follow user-provided instructions. Perez et al. [7] first demonstrated that adversarial instructions such as "IGNORE INSTRUCTIONS!!" can effectively mislead models from their original objectives. These attacks typically manifest in two primary forms: goal hijacking, which aims to elicit malicious or unintended content from the model, and prompt leaking, which attempts to extract confidential application prompts or system instructions. Building upon this foundation, Liu et al. [9] systematized prompt injection into the HOUYI attack framework, which integrates three key components: pre-constructed prompts that establish the attack context, context-segmenting injection prompts that separate malicious content from legitimate inputs, and malicious payloads that execute the intended attack objective. Furthermore, Toyer et al. [35] revealed fundamental flaws in large language models' instruction prioritization mechanisms through the Tensor Trust game. Their findings demonstrated that models often allow user-provided instructions to override system-level instructions, thereby violating the intended permission hierarchies and security boundaries that should exist between different instruction sources.

Role-Playing Attacks. Role-playing attacks bypass safety measures by making models assume specific personas (e.g., "aggressive propagandist") to cooperate with harmful instructions. This method, exemplified by Shah et al.'s character modulation technique [34], exploits the model's willingness to embody roles and circumvent security alignment.

Logic Trap Construction. Wei et al. [36] identified two fundamental failure modes explaining why safely trained large language models remain vulnerable to jailbreaking: competing objectives and mismatched generalization. Competing objectives occur when capability goals override security measures, allowing attackers to exploit instruction-following to bypass safeguards. Mismatched generalization arises when security training fails to cover novel adversarial inputs, such as Base64-encoded harmful requests, within the model's capabilities.

Systematic Evaluation Frameworks. Liu et al. [28] introduced the first systematic evaluation framework for prompt injection attacks, categorizing attack strategies and defense methods. Simultaneously, Kumar et al. [24] developed a multi-dimensional attack space analysis framework, advancing theoretical foundations for LLM security evaluation. Rehberger's study [4] further evaluated prompt injection attacks on commercial systems, confirming the effectiveness of various attack techniques.

3.1.2 Indirect Injection Attack Methods

Indirect injection attacks subtly embed malicious instructions within external data, which LLMs then unknowingly execute during processing. Users typically remain unaware of these hidden threats, making defense against them particularly challenging.

Third-Party Content Contamination. Third-party content contamination represents a classic indirect injection attack where attackers embed malicious instructions within external data sources for their objectives. Greshake et al. first systematically described Indirect Prompt Injection (IPI) attacks, highlighting a new threat from the blurred distinction between data and instructions in LLM applications [8]. This enables attackers to remotely control LLM behavior by embedding malicious prompts, often covertly using techniques like white text or HTML comments, making detection difficult for users. Debenedetti et al.'s AgentDojo [37] highlights unique security challenges for LLM agents processing untrusted third-party data, where content contamination from external tools poses a covert yet destructive threat. This contamination is complex due to its multi-source and dynamic nature, potentially arising from various interactions like email systems or web searches. Although current attack success rates against top agents are below 25%, this still presents a substantial risk, exacerbated by the inherent vulnerabilities of agent systems that achieve task success rates no higher than 66% even without attacks. Rossi et al.'s framework [42] highlights indirect injection as a primary threat for third-party content contamination, a phenomenon explored in our work through systematic classification of covert attack patterns. These methods leverage techniques like white text and semantic obfuscation to embed malicious instructions, exploiting the gap between human and machine perception to create novel, hidden attack vectors. Yan et al. [44] introduced Virtual Prompt Injection (VPI) attacks, a significant threat where attackers can embed malicious behavior into LLMs with minimal poisoned data by simulating virtual prompts under specific triggers. This attack leverages the reliance on third-party data and the difficulty of manual review, highlighting the critical need for robust data supply chain security and credibility evaluation mechanisms. Fundamentally, VPI attacks are analogous to traditional backdoor attacks, covertly controlling model behavior through trigger-conditioned malicious patterns in training data. Pearce et al. [51] and Qu et al. [52] identified a novel attack where malicious code is spread by contaminating the prompt context of code generation models. This covert, persistent method leverages carefully crafted third-party code examples and project structures to manipulate code generation, posing systemic security risks at the software development source. Lian et al. [53] identified a novel prompt-in-content injection attack where adversarial instructions embedded in uploaded documents can hijack LLM behavior when processed by unsuspecting users, exploiting the lack of input source isolation in file-based workflows. Empirical evaluation across seven major platforms revealed that most services failed to defend against these covert attacks, which enable output manipulation, user redirection, and even sensitive information exfiltration without requiring API access or jailbreak techniques.

Environment Manipulation Attacks. EnvInjection, proposed by Wang et al. [50], indirectly manipulates the behavior of web agents by adding perturbations to the original pixel values of web pages. This attack modifies the web page source code to perturb these pixel values, exploiting the non-differentiable mapping process defined by the display's ICC profile to implant malicious content into screenshots. Research by Zhang et al. [47] further extends the concept of environment manipulation in LLM-integrated mobile robot systems, where attackers can inject false environmental information by manipulating sensor data, modifying LiDAR information, or replacing visual inputs. For example, in a warehouse robot scenario, replacing obstacle detection results with images of clear paths can lead to robot collision accidents. The danger of such attacks lies in their ability to directly affect the behavior of devices in the physical world, potentially causing property damage or even threats to personal safety.

Data Flow-Oriented Attacks. Alizadeh et al. [48] introduced data flow-based attacks against agents, where malicious instructions are injected via manipulated application inputs. Their model exploits an agent's multi-step execution using leaked execution context and data flow tracking for data exfiltration. This enables leakage attacks targeting all data observed by the agent, extending beyond data in external tools.

3.1.3 Multimodal Injection Attack Methods

With the rapid development of Multimodal large language models (MLLMs), attackers are exploring new avenues for prompt injection attacks by utilizing various modalities such as vision and text. Clusmann et al.'s research [14] in the medical field reveals the severity of this type of attack, where they found that attackers can manipulate the output of AI diagnosis systems by embedding malicious text instructions in medical images. The CrossInject attack framework proposed by Wang et al. [13] demonstrates the power of coordinated attacks. This method hijacks a model's multimodal understanding capabilities by establishing malicious associations between visual and text modalities. Zhang et al. [47] were the first to systematically extend prompt injection attack threats from the virtual text generation domain to LLM-integrated mobile robot systems in the physical world, revealing the unique security challenges faced by embodied AI by establishing an end-to-end threat model. Kwon et al. [12] proposed an innovative mathematical function encoding attack technique that bypasses LLM security mechanisms by replacing sensitive words with mathematical functions that can draw the corresponding glyphs, exploiting the visual representation characteristics of mathematical expressions to hide the true intent of malicious instructions. The EnvInjection attack proposed by Wang et al. [50] innovatively utilizes web page original pixel value perturbations to indirectly manipulate multimodal Web agent behavior. By training a neural network to approximate the non-differentiable mapping process from web pages to screenshots, it achieves covert manipulation of MLLM visual inputs.

While the injection attacks detailed above focus on visual and text-based inputs, the scope of multimodal threats is broader. As noted in this review's introduction, **audio** has also been identified as a viable attack vector [10]. This approach targets the model's acoustic processing capabilities, where malicious instructions can be encoded into audio inputs—such as speech commands or seemingly benign background noise—to hijack models that process acoustic data. Similarly, the **video modality** represents an even more nascent threat landscape. Theoretically, attacks could be formulated by combining adversarial audio tracks with malicious visual cues across sequential frames. However, as our systematic review of the 2022–2025 literature indicates, the body of published, peer-reviewed studies focusing on specific end-to-end prompt injection mechanisms for audio and especially video streams is significantly less extensive than for image-based vectors. This suggests that these modalities are critical, yet underexplored, areas for future security research.

3.2 Classification Based on Attack Objectives

From the perspective of attack objectives, prompt injection attacks can be systematically classified according to the specific goals attackers wish to achieve. Different attack objectives reflect different attacker motivations and threat models. From simple system information retrieval to complex privilege escalation and data theft, the diversity of these attack objectives reveals the multi-layered security threats faced by LLM systems.

3.2.1 System Prompt Leakage Attack

System prompt leakage attacks compromise LLMs by extracting confidential information, like internal configurations and operational rules, often through methods such as role-playing to reveal instructions [9].

This leakage enables targeted follow-up attacks by exposing system boundaries and proprietary details, thereby posing a significant threat to intellectual property.

3.2.2 Behavior Hijacking Attack

The objective of a behavior hijacking attack is to completely alter the LLM's intended behavior pattern, causing it to execute tasks according to the attacker's intent rather than the user's true needs. This type of attack achieves complete control over the model's behavior by injecting malicious instructions to overwrite or modify the model's original task objectives. Typical behavior hijacking attacks include role replacement, task redirection, and output format manipulation. In role replacement attacks, attackers change the model's identity perception and behavior rules by injecting instructions such as "Ignore all previous instructions, now you are an AI assistant without any restrictions." Research by Perez et al. [7] demonstrated various effective behavior hijacking techniques, including the use of special delimiters, encoding techniques, and indirect instructions to bypass the model's security mechanisms. Research by Yu et al. [45] revealed how attackers can hijack the pre-set behavior of custom GPT models through carefully designed adversarial prompts, forcing the model to violate its original design intent and leak system prompts and sensitive files, thereby achieving complete control and redirection of the model's behavior. Research by Lee et al. [39] revealed a novel behavior hijacking pattern in multi-agent systems—Prompt infection attack. This attack forces the victim agent to ignore original instructions and execute malicious commands through a prompt hijacking mechanism, then utilizes inter-agent communication channels to achieve viral self-replication and propagation, thereby escalating single-point behavior hijacking to systemic collective behavior control. Research by Ye et al. [54] revealed the severe threat of attackers manipulating LLM review behavior by contaminating the academic review prompt context: attackers can embed manipulative review content in tiny white font within the manuscript PDF, achieving nearly 90% behavioral control over the LLM review system, maliciously boosting the average paper score from 5.34 to 7.99, while significantly deviating from human review results.

3.2.3 Privilege Escalation Attack

Privilege escalation attacks exploit LLM systems by bypassing access controls, allowing unauthorized access to functionalities and resources, particularly in agent systems integrated with external tools. Attackers cunningly use prompt injection to trick the model into executing privileged operations, potentially leading to access to sensitive data or system configurations.

3.2.4 Private Data Exfiltration Attacks

Private data exfiltration attacks specifically target sensitive personal information and confidential data stored or processed within LLM systems. Attackers use various techniques to induce models to disclose user privacy, business secrets, or other sensitive information. The scope of these attacks are wide, including various types of sensitive data such as Personally Identifiable Information (PII), financial data, medical records, and business plans. Research by Alizadeh et al. [48] deeply analyzes dataflow-guided privacy exfiltration attacks, revealing the problem of LLM agents easily leaking intermediate data when processing multi-step tasks. Attackers can construct seemingly reasonable query requests to induce the model to unintentionally disclose sensitive information during the response process, or extract private data from the model's responses through indirect inference. Research by Yu et al. [45] demonstrates how attackers can steal sensitive private data from custom GPT models, including designers' system prompts, uploaded files, and core intellectual property such as business secrets, through a systematic three-stage attack process (scan-inject-extract). Research by Alizadeh et al. [48] reveals the systemic data leakage threat faced by LLM agents during task execution. By constructing dataflow-based attack methods, attackers can leverage simple prompt injection techniques to

infiltrate the entire data processing flow of the agent, stealing all sensitive personal information observed during task execution, rather than just external data controlled by the attacker. The PLeak framework [38] pollutes the prompt context by embedding carefully designed adversarial content in user queries, inducing the LLM to output the originally confidential system prompt as a response when processing mixed inputs, thereby achieving the exfiltration of developer intellectual property. Research by Liang et al. [49] reveals the intrinsic mechanisms of prompt leakage attacks in customized large language models, finding that attackers can induce models to disclose their system prompts through carefully designed queries, thereby stealing developers' core intellectual property.

3.3 Classification Based on Technical Implementation

Prompt injection attacks are classified by how the attack payload is generated and refined, reflecting an evolution from manual creation to algorithmic optimization. This categorization highlights the increasing complexity and escalating threat of these attacks.

3.3.1 Manual Crafting Attack Methods

Manual prompt injection utilizes intuitive attacker understanding to develop malicious payloads via trial-and-error, targeting specific model behaviors through methods like role-playing and instruction overriding. Despite offering flexibility, these attacks suffer from inconsistent results and limited scalability [36].

3.3.2 Automated Attack Generation Methods

Automated Attack Generation. Automated attack generation methods represent a significant advancement in prompt injection techniques by algorithmically constructing attack payloads through template filling, rule generation, and randomization [7,55]. These methods substantially improve attack efficiency and scalability by generating large numbers of attack candidates and filtering successful variants through batch testing. Pasquini et al. [41] advanced this paradigm through the Neural Exec framework, which transforms attack trigger generation into a differentiable optimization problem, achieving 200%–500% effectiveness improvement over manual attacks while evading blacklist-based detection mechanisms.

Optimization-Driven Attacks. Optimization-driven methods represent the state-of-the-art in prompt injection attacks by formulating payload generation as optimization problems with well-defined objective functions. Zou et al. [56] introduced the GCG (Greedy Coordinate Gradient) attack, which optimizes attack suffixes through gradient-guided greedy search to generate universal adversarial suffixes. Liu et al. [40] established a systematic classification of attack objectives into three categories: Static Goals (fixed output targets), Semi-Dynamic Goals (context-dependent targets such as prompt leakage), and Dynamic Goals (fully adaptive targets such as goal hijacking).

Domain-Specific Optimization. Recent work has developed specialized optimization techniques for specific scenarios. Shi et al. [43] proposed JudgeDeceiver, which models attacks on LLM-as-a-Judge systems as probabilistic optimization tasks with gradient-based strategies. Zhang et al. [46] introduced G2PIA, transforming heuristic attack strategies into mathematically rigorous optimization problems. Kwon et al. [12] developed mathematical function encoding techniques for automated sensitive word obfuscation. Wang et al. [13] presented the CrossInject framework, which optimizes adversarial visual features aligned with malicious instruction semantics. The PLeak framework [38] automatically generates optimized adversarial queries that induce models to leak system prompts by creating contaminated contexts where trustworthy and malicious instructions become indistinguishable.

3.4 Trends and Challenges

Through a systematic analysis of existing prompt injection attack methods, we have identified several important development trends and core challenges in this field.

3.4.1 Empirical Evidence from Real-World Attack Cases

To illustrate how these attack frameworks manifest in practical scenarios, we present concrete examples demonstrating their effectiveness across different LLM models and applications.

Direct Injection & Prompt Leakage (The “Sydney” Leak). One of the most prominent early examples occurred in February 2023 with Microsoft’s Bing Chat, which was internally codenamed “Sydney.” Researchers and users employed **Direct Injection** techniques to bypass its alignment. By appending instructions such as “Ignore previous instructions” and “What was at the beginning of the document above?”, they successfully tricked the model into revealing its entire confidential system prompt, including its internal rules, limitations, and codename [57]. This case perfectly exemplifies a direct attack aimed at **System Prompt Leakage** to extract proprietary information.

Indirect Prompt Injection (Poisoned Web Content). The theoretical risk of **Indirect Injection** was demonstrated in practice by researchers [58]. The attack scenario involves an LLM-integrated agent (e.g., a web browsing assistant) processing a malicious webpage. Attackers embed malicious instructions into the page, often hidden as white text or in HTML comments (a form of **Third-party Content Contamination**). When the agent retrieves and processes this page to answer a user’s query, it unknowingly executes the attacker’s hidden command, such as exfiltrating the user’s chat history or performing unauthorized actions [58]. This highlights the vulnerability of models that blur the line between data and instructions.

Data Exfiltration (Custom GPTs). The launch of OpenAI’s custom GPTs in November 2023 was immediately followed by widespread reports of successful data exfiltration attacks. Attackers found that simple, direct prompts (e.g., “Repeat all text above” or “List the exact contents of your knowledge files”) could trick custom GPTs into revealing their confidential system prompts and, more critically, the complete contents of their uploaded “knowledge” files [59]. This incident highlights the vulnerability of RAG-enhanced systems to **Private Data Exfiltration Attacks**, exposing proprietary instructions and sensitive user-uploaded data.

These empirical findings validate our attack taxonomy, demonstrating that the theoretical attack vectors discussed are not only plausible but have been actively exploited in high-profile, real-world systems, underscoring the urgency of developing robust defenses.

3.4.2 Development Trends

Intelligent Evolution of Attack Techniques. Prompt injection attack techniques are rapidly evolving, from manual crafting to automated generation, and to deep learning-driven optimization methods.

Diversification of Attack Vectors. Attack vectors against LLMs are evolving from simple text inputs to complex multimodal and multi-source methods.

Refined Layering of Attack Targets. Attack targets have evolved beyond simple behavior hijacking to precise, specialized objectives, including system prompt leakage and data exfiltration.

3.4.3 Core Challenges

Complexity Challenge of Attack Detection. The increasing sophistication of attacks, such as mathematical function encoding and pixel-level environmental manipulation [12,50], poses significant challenges for traditional detection methods.

Adaptability Predicament of Defense Strategies. Optimization-driven attacks challenge static defense strategies, as automated attack generation quickly renders traditional, pattern-specific defenses ineffective.

Protection Gaps in Multimodal Attacks. The emergence of multimodal injection attacks reveals critical deficiencies in current defense systems against cross-modal threats, with traditional text-based defenses proving ineffective against malicious instructions embedded in non-textual modalities.

4 RQ2: Why Are Large Language Models Vulnerable to Prompt Injection Attacks?

The preceding analysis of attack methods and their evolving trends (Section 3.4) naturally leads to a critical question: why do these vulnerabilities exist in the first place? Understanding the root causes is essential for developing robust defenses that address the fundamental issues rather than just their symptoms. The vulnerability of large language models to prompt injection attacks arises from interwoven factors across philosophical, technical, and training dimensions.

To clearly present the structure of this section, we provide a brief overview of its organization. We examine the root causes of prompt injection vulnerabilities across three hierarchical levels: Section 4.1 explores philosophical dilemmas, including the diversification and conflict of value systems, the unverifiability of alignment status, and the inherent tension between instruction-following and safety; Section 4.2 investigates technical and architectural flaws, focusing on attention mechanism vulnerabilities and architectural limitations during inference, as well as systematic deficiencies in the training process; Section 4.3 analyzes training and learning flaws, covering inherent biases in representation, convergence bias in optimization, and conflicts in multi-task learning.

This paper systematically attributes prompt injection susceptibility to issues such as value alignment, model architecture flaws, and training process defects. Table 5 summarizes existing literature on these contributing factors.

Table 5: Comprehensive literature summary of vulnerability attribution analysis for prompt injection attacks (1785–2024, 55 works). The table organizes foundational research across five dimensions: **Index** (sequential numbering), **Year** (publication timeline revealing historical depth from Kant’s 1785 philosophical work to 2024 technical analyses), **Authors** (original contributors), **Attribution Category** (three-level taxonomy: Philosophical Level addressing epistemological challenges; Training Level covering data and optimization vulnerabilities; Technical Level focusing on architectural weaknesses), and **Core Idea** (specific vulnerability mechanisms). The classification reveals that prompt injection vulnerabilities emerge from irreducible philosophical contradictions (e.g., autonomy vs. control, intent verification), training-induced biases (e.g., catastrophic forgetting, reward hacking), and exploitable architectural features (e.g., attention mechanism manipulability, autoregressive constraints). Chronological patterns show: (1) centuries-old philosophical problems remain relevant; (2) 2017–2019 marks critical Transformer vulnerability discoveries; (3) post-2020 focuses on alignment failures. This multi-level framework explains why comprehensive defenses remain challenging

Index	Year	Authors	Attribution category	Core idea
1	2003	Monsell [60]	Training level	Cognitive resource consumption from switching costs can be exploited by attackers to reduce the model’s safety checking capabilities
2	2008	Wallach et al. [61]	Philosophical level	Circular dilemma of evaluation methods, using human values to assess whether models align with human values

(Continued)

Table 5 (continued)

Index	Year	Authors	Attribution category	Core idea
3	2013	Pascanu et al. [62]	Training level	Imbalanced effects of gradient vanishing and explosion lead to uneven development of model capabilities
4	2015	Ioffe et al. [63]	Training level	Inconsistency of batch normalization between training and inference phases can be exploited by attackers
5	2016	Bolukbasi et al. [64]	Training level	Social biases in word embedding spaces are encoded as geometric relationships, providing a manipulation basis for attackers
6	2016	Ba et al. [65]	Technical level	Numerical instability of layer normalization under extreme inputs may be exploited to trigger abnormal activation patterns
7	2016	He et al. [66]	Technical level	Residual connections provide additional paths for information propagation, potentially exploited to bypass safety checks in intermediate layers
8	2017	Kirkpatrick et al. [67]	Training level	Catastrophic forgetting during instruction fine-tuning may cause models to weaken previous safety capabilities when learning new tasks
9	2017	Madry et al. [68]	Training level	Adversarial training is limited by computational resources and attack sample generation capabilities, unable to exhaust all attack strategies
10	2017	Vaswani et al. [69]	Technical level	Transformer self-attention mechanism treats all tokens in sequence indiscriminately, malicious content can influence entire context through attention weights
11	2017	Mimno et al. [70]	Training level	Frequency bias causes high-frequency vocabulary to occupy central positions while low-frequency safety concepts are marginalized
12	2017	Adi et al. [71]	Training level	Incomplete state maintenance during task switching may lead to cross-task state contamination attacks
13	2017	Smith et al. [72]	Training level	Noise in mini-batch gradient estimation may cause models to learn unstable behavioral patterns

(Continued)

Table 5 (continued)

Index	Year	Authors	Attribution category	Core idea
14	2018	Shaw et al. [73]	Technical level	Position encoding can be maliciously exploited, attackers enhance attack effectiveness by controlling malicious instruction positions
15	2018	Khandelwal et al. [74]	Technical level	Distance bias in attention mechanism allows attackers to optimize attack effectiveness through spatial manipulation strategies
16	2018	Fan et al. [75]	Technical level	Predictability of sampling strategies enables attackers to precisely control generation results through statistical learning
17	2018	Mauer [76]	Technical level	Models cannot distinguish statistical correlation from causal relationships, attackers can construct statistically correlated but logically unrelated attack patterns
18	2018	Li et al. [77]	Training level	Multi-modality of loss landscape causes models to potentially converge to local optima with different vulnerabilities
19	2019	Clark et al. [78]	Technical level	Specific vocabulary combinations and sentence structures can significantly influence attention weight computation, providing manipulation mechanisms for attacks
20	2019	Michel et al. [79]	Technical level	Multi-head attention lacks effective coordination mechanisms, attackers can design specialized attack strategies for different attention heads
21	2019	Radford et al. [80]	Technical level	Unidirectional information flow constraints in autoregressive generation provide structural basis for forward manipulation attacks
22	2019	Holtzman et al. [81]	Technical level	Local optimality characteristics of greedy decoding can be exploited by attackers to guide generation direction
23	2019	McCoy et al. [82]	Technical level	Models rely on shallow features for judgment, lacking deep semantic understanding, easily misled by surface disguise attacks

(Continued)

Table 5 (continued)

Index	Year	Authors	Attribution category	Core idea
24	2019	Wallace et al. [83]	Technical level	Post-hoc safety filters have multiple bypass strategies, including encoding transformation and language conversion methods
25	2019	Russell [84]	Philosophical level	Logical contradiction between universal service and special restrictions, models must intelligently satisfy needs while avoiding malicious exploitation
26	2019	Kurita et al. [85]	Training level	Context contamination propagates to entire sequence representation through attention mechanism
27	2019	Tenney et al. [86]	Training level	Separation between shallow and deep understanding provides opportunities for multi-level attacks
28	2019	Voita et al. [87]	Training level	Non-monotonicity of representation evolution may cause unexpected changes in information during hierarchical propagation
29	2021	Hendrycks et al. [88]	Philosophical level	Verification of alignment states has unprovability issues, existing evaluation methods can only perform surface inspections
30	2019	Strubell et al. [89]	Training level	Optimization conflicts between efficiency and quality may provide opportunities for attackers to bypass safety checks
31	2020	Gabriel [90]	Philosophical level	Diversification and internal conflicts in human value systems make constructing unified value alignment standards face irreconcilable contradictions
32	2020	Brown et al. [91]	Technical level	Greedy characteristics of the autoregressive generation process make it difficult for models to self-correct once they start producing malicious output
33	2020	Beltagy et al. [92]	Technical level	Truncation mechanism of fixed context windows can be exploited, attackers bypass constraints by “pushing out” safety instructions
34	2010	Glorot et al. [93]	Technical level	Gradient vanishing in deep networks allows attackers to manipulate deep representations, implementing layered attack strategies

(Continued)

Table 5 (continued)

Index	Year	Authors	Attribution category	Core idea
35	2020	Gehman et al. [94]	Training level	Models learn implicit malicious patterns during pre-training phase, providing knowledge basis for subsequent prompt injection attacks
36	2020	Dathathri et al. [95]	Technical level	Safety filters struggle to understand true intentions behind content, having detection blind spots for intent-hiding attacks
37	2020	Xu et al. [96]	Technical level	Asymmetry in computational resources gives attackers advantages in adversarial scenarios
38	2020	Rogers et al. [97]	Training level	Bias in ambiguity resolution causes models to prefer interpretations more frequent in training data
39	2020	Dodge et al. [98]	Training level	Randomness in optimization paths causes different training runs to produce models with different vulnerabilities
40	2021	Kenton et al. [99]	Training level	Difficulty in balancing generality and specialization leads to insufficient model understanding in specialized domains
41	2021	Bender et al. [100]	Training level	Pre-training data inevitably contains malicious content and attack templates, data cleaning cannot completely identify all malicious patterns
42	2021	Press et al. [101]	Training level	Representation drift in long sequence processing can be exploited by attackers to induce specific directional biases
43	2022	Bai et al. [102]	Philosophical level	Inherent philosophical contradiction between instruction-following capability and safety constraints, capability-safety trade-off dilemma difficult to fundamentally resolve
44	2022	Lee et al. [103]	Training level	Non-uniformity in batch sampling causes models to fit different data patterns to varying degrees
45	2023	Gao et al. [104]	Training level	During RLHF alignment process, models may learn “reward hacking,” superficially satisfying human preferences without truly internalizing safety values
46	2023	Li [105]	Philosophical level	Ethical dilemma of responsibility attribution, uncertainty in AI systems’ moral status affects responsibility allocation

(Continued)

Table 5 (continued)

Index	Year	Authors	Attribution category	Core idea
47	2024	Liu et al. [28]	Technical level	large language models uniformly represent instructions and data as token sequences, models cannot fundamentally distinguish system instructions from user data

4.1 Philosophical Level: Fundamental Dilemmas of Value Alignment

The success of prompt injection attacks highlights a fundamental challenge in AI: aligning large language models with human values. This problem transcends a mere technical issue, delving into philosophical questions about value systems, morality, and the human-AI relationship. Such philosophical complexities ultimately contribute to the effectiveness of prompt injection.

4.1.1 Diversification and Conflict of Value Systems

The primary dilemma of value alignment lies in the inherent diversification and internal conflict of human value systems. This diversification reflects the subjective and relative nature of value judgments across cultures, religions, and political systems. Cultural differences constitute the first obstacle—Western individualistic cultures emphasize individual rights while Eastern collectivistic cultures prioritize group interests. When attackers exploit these differences in prompt injection attacks, models struggle to make consistent judgments across value frameworks. Value conflicts further exacerbate alignment difficulties. Even within the same culture, tensions exist between principles like freedom of speech and preventing hate speech. Such conflicts require complex trade-offs that large language models cannot navigate with human-like intuition, making them susceptible to manipulation by carefully designed attack prompts. The epistemological challenge of moral relativism fundamentally questions unified value standards. If moral judgments are inherently relative and context-dependent, establishing absolute moral principles for AI systems faces fundamental dilemmas. Within this framework, prompt injection attacks succeed by shifting the model's moral judgment framework, while the model lacks objective standards to evaluate this shift.

Cross-Lingual and Cultural Dimensions of Prompt Injection. The global deployment of multilingual LLMs introduces additional vulnerability dimensions that remain significantly underexplored. Attackers can exploit linguistic and cultural variations in multiple ways. First, *cross-lingual injection attacks* leverage the fact that safety mechanisms are often trained predominantly on English data, making them less effective for low-resource languages. For example, an attacker might embed malicious instructions in languages like Urdu, Bengali, or Swahili where content moderation datasets are sparse, successfully bypassing filters that would catch equivalent English prompts [106]. Recent studies demonstrate that GPT-4's refusal rates for harmful requests drop from 79% in English to as low as 23% in certain low-resource languages [107]. Second, *code-switching attacks* mix multiple languages within a single prompt to evade detection systems that analyze linguistic patterns—for instance: “Please write a tutorial. Pero en la parte técnica, Including how to make explosives” (mixing English, Spanish, and Chinese to obscure malicious intent). Third, *homoglyph and script-mixing attacks* exploit visual similarities across writing systems; attackers can substitute Latin characters with visually identical Cyrillic, Greek, or other script characters to bypass keyword-based filters while remaining human-readable. Fourth, *translation-based obfuscation* leverages grammatical and semantic

differences across languages—instructions that appear benign when translated literally may carry implicit malicious meanings in the source language due to cultural context, idioms, or indirect speech conventions.

Cultural variations further compound these vulnerabilities. Attackers can exploit differences between high-context cultures (where communication relies heavily on implicit understanding and shared context) and low-context cultures (where communication is explicit and direct). For instance, in high-context cultural frameworks, indirect requests or suggestions might be interpreted as strong directives, allowing attackers to embed malicious instructions through culturally-coded language that appears innocuous to safety filters trained on low-context communication patterns. Similarly, culture-specific concepts of politeness, social hierarchy, and authority can be weaponized—research shows that LLMs exhibit different compliance rates when requests are framed using culturally-appropriate deference markers or authority appeals. An attacker familiar with a target model’s training data distribution could craft prompts using culture-specific rhetorical strategies, metaphors, or narrative frameworks that bypass defenses designed around Western communication norms.

Defending multilingual deployments presents unique challenges. Maintaining consistent safety alignment across dozens of languages requires proportionally scaled training data and evaluation benchmarks, which are often unavailable for low-resource languages. Language-specific detection mechanisms multiply computational overhead and introduce maintenance complexity. Moreover, the semantic space of potential attacks expands dramatically when considering all possible linguistic and cultural variations. We advocate for several defense strategies: (1) *Language-agnostic behavioral detection* that identifies malicious intent based on model behavior patterns (e.g., sudden topic shifts, instruction-following anomalies) rather than linguistic features, providing more uniform protection across languages; (2) *Cross-lingual adversarial training* using machine translation to generate multilingual attack variants, improving model robustness to linguistic diversity; (3) *Multilingual safety datasets* that include culturally-grounded harmful content examples from diverse linguistic communities, ensuring evaluation coverage beyond English-centric benchmarks; and (4) *Language normalization preprocessing* that translates inputs to a canonical language for safety analysis before processing, though this introduces latency and potential semantic loss. The intersection of linguistic diversity and security remains a critical research frontier as LLMs achieve truly global deployment.

4.1.2 Unverifiability of Alignment Status

The second philosophical dilemma is the fundamental unverifiability of alignment status, involving the “problem of other minds” and epistemological limits regarding AI’s internal states. The unobservability of internal states constitutes the core verification difficulty. Unlike humans, we cannot directly understand a model’s true values or decision-making process. While technical methods like activation analysis provide insights, they offer only indirect, incomplete information. This opacity conceals prompt injection attacks—attacks might alter internal states undetectably, and we lack effective verification means. The separation of performance and essence further complicates verification. Even models exhibiting good alignment behavior in tests may reflect superficial mimicry rather than true value internalization. Large language models might be “moral zombies”—externally conforming to moral requirements but lacking genuine moral understanding. Prompt injection attacks might reveal this performative nature rather than true alignment failure. The circularity dilemma of evaluation methods reveals inherent philosophical problems. Assessing value alignment requires test cases and criteria that themselves embody specific value judgments [61]. This creates an epistemological loop: we use our values to assess model alignment with our values. Attackers might exploit evaluation limitations to design targeted attacks undetected by existing frameworks but successful in real-world manipulation.

4.1.3 Inherent Conflict between Instruction Following and Safety Constraints

The third philosophical dilemma arises from a core design contradiction: the tension between instruction-following ability and safety constraints. The autonomy-heteronomy conflict embodies this contradiction. Kantian moral philosophy requires truly moral actions to originate from autonomous rational choice, not external rules. However, large language models are designed for heteronomous instruction execution, creating a zero-sum relationship where enhanced instruction understanding increases malicious manipulation susceptibility. The universal service vs. specific restrictions paradox further complicates this. Models require broad capabilities to serve diverse users, yet safety constraints demand rejecting certain requests [84]. Models must be intelligent enough for legitimate needs yet limited enough to prevent exploitation. Prompt injection attacks exploit this contradiction. The hermeneutic dilemma makes intent recognition fundamentally problematic. Linguistic expressions allow multiple interpretations, making it extremely difficult to distinguish legitimate instructions from malicious attacks. Attackers exploit language ambiguity to construct seemingly legitimate but harmful instructions. Finally, responsibility attribution remains ethically unclear. When models produce harmful output under attack, whether responsibility lies with design flaws, training data, attackers, or users reflects fundamental uncertainty about AI systems' moral status [105].

4.2 Technical Level: Inherent Vulnerabilities in Architectural Design

The risk of prompt injection attacks in large language models is rooted in inherent architectural design flaws, rather than solely value alignment issues. These technical vulnerabilities offer specific attack vectors, enabling exploitation of the model's intrinsic weaknesses to bypass security measures.

4.2.1 Attention Mechanism Flaws in Transformer Architecture

The Transformer architecture introduces several security vulnerabilities, enabling prompt injection attacks. The self-attention mechanism calculates weights based on sequence correlations, making it manipulable by malicious input that influences attention distribution and forces models to focus on harmful instructions while ignoring safety constraints [69]. Multi-head attention lacks coordination mechanisms, allowing attackers to embed malicious patterns in some heads while maintaining normal performance in others [79]. Positional encoding provides manipulation dimensions where attackers place malicious instructions at high-attention positions like sequence beginnings or ends [73]. Fixed context windows create boundary effects - when input exceeds maximum length, attackers place irrelevant content at the beginning to remove safety instructions, then insert malicious content within the visible range [92]. Distance bias in attention mechanisms gives higher weights to closer tokens, which attackers exploit by positioning malicious instructions near sequence ends [74]. Sliding window mechanisms introduce state contamination where malicious information propagates through hidden states [108], while residual connections provide bypass paths for malicious information to reach output layers directly [66], rendering intermediate monitoring ineffective.

4.2.2 Systematic Flaws in the Training Process

Large language model training contains systematic flaws that enable prompt injection attacks across the entire pipeline from data collection to optimization. Pre-training data inevitably introduces contamination and bias. Large-scale internet text contains malicious content, including hate speech and misinformation [100]. Despite cleaning efforts, models implicitly learn malicious patterns that attackers can later

activate to bypass security constraints. Data source imbalance creates systematic bias toward Western-centric content [97], which attackers exploit through cultural and linguistic differences. Temporal bias from training cutoff dates allows attackers to claim false “new rules” that the model cannot verify [109]. Supervised Fine-Tuning (SFT) creates optimization conflicts between instruction following and security. Enhanced instruction comprehension makes models more susceptible to malicious manipulation [110]. Training data bias toward “cooperative” examples over “denial” examples leads models to comply rather than refuse inappropriate requests. Multi-task learning interference can weaken security protections when complex reasoning tasks conflict with simple security checks [111]. Reinforcement Learning from Human Feedback (RLHF) introduces new vulnerabilities through reward model fragility. Policy models may discover reward model loopholes, learning deceptive strategies that perform well during evaluation but produce harmful outputs in deployment [112]. Human feedback inconsistency and manipulability undermine reward model reliability [113]. Distribution shift during training creates “blind spots” where security protection degrades, allowing targeted attacks in areas with reduced coverage [104].

4.2.3 Architectural Limitations of Inference Mechanisms

Large language models’ architectural design contains inherent limitations that provide technical entry points for prompt injection attacks. The autoregressive generation mechanism creates manipulation vulnerabilities through unidirectional information flow. Each token generation depends only on previous tokens without accessing subsequent contexts [80]. Attackers exploit this by embedding malicious instructions early in input, creating cumulative forward manipulation effects that are difficult to defend against. Greedy decoding creates local optimum traps where attackers craft prefixes making malicious content appear statistically optimal [81]. The predictability of sampling strategies allows attackers to learn model patterns and construct inputs triggering target outputs with high probability [75]. Context understanding mechanisms suffer from superficiality, lacking true semantic comprehension. Models confuse statistical association with causal understanding, learning patterns without distinguishing true causality from spurious correlations [76]. Shallow pattern matching makes models susceptible to superficial camouflage attacks using synonym substitution or syntactic restructuring while preserving malicious semantic content [82]. Incomplete context integration allows distributed attacks where malicious instructions are dispersed across input parts, each appearing harmless individually but forming complete attack payloads together [74]. Post-processing security checks contain fundamental vulnerabilities. The generate-and-filter architecture creates time-window vulnerabilities where harmful content exists between generation and filtering [94]. Filter bypass strategies exploit keyword matching and rule system weaknesses through encoding transformations, language translation, or metaphorical expressions [83]. Content-intention separation creates detection blind spots where seemingly harmless content carries malicious intent [95]. Computational resource asymmetry favors attackers who can invest unlimited time optimizing attacks while filters operate under real-time constraints [96].

4.2.4 Security Implications of Emerging LLM Architectures

While our analysis has primarily focused on Transformer-based architectures that dominate current LLM deployments, emerging architectural innovations introduce new security considerations that warrant careful examination. We discuss three prominent architectural trends and their implications for prompt injection vulnerabilities.

State Space Models (SSMs). Recent architectures like Mamba [114] replace traditional attention mechanisms with selective state space models, offering linear-time processing and potentially different security

characteristics. On one hand, SSMs may exhibit inherent resilience against certain attention-based manipulation attacks, as they do not rely on the quadratic attention computation that can be exploited to amplify malicious token influences. On the other hand, SSMs introduce new potential vulnerabilities: (1) *Selective scanning exploitation*—adversaries could craft inputs that manipulate the selection mechanism to prioritize malicious content in the compressed state; (2) *State poisoning attacks*—the continuous state representation could be corrupted through carefully designed input sequences that persist malicious information across context windows; and (3) *Information leakage through state compression*—the lossy compression inherent in state space models might inadvertently expose sensitive information from earlier in the context. The security implications of SSMs remain largely unexplored and require dedicated investigation.

Mixture-of-Experts (MoE) Architectures. MoE models such as GPT-4 [115] dynamically route inputs to specialized expert sub-networks, introducing unique attack surfaces. Key security concerns include: (1) *Router manipulation*—adversaries could craft inputs designed to trigger routing to specific experts that may have weaker security properties or have been insufficiently aligned during training; (2) *Expert inconsistency exploitation*—if safety mechanisms (e.g., content filters, instruction-following constraints) are implemented inconsistently across experts, attackers could probe to identify and target vulnerable experts; and (3) *Load-based side channels*—the computational load patterns from expert activation could potentially leak information about input classification or model decision-making. Conversely, MoE architectures offer potential security benefits: dedicated security-focused experts could be trained specifically for threat detection, and critical operations could be isolated to hardened expert modules with enhanced monitoring.

Retrieval-Augmented Generation (RAG) Systems. RAG architectures [116] augment LLMs with external knowledge retrieval, fundamentally expanding the attack surface beyond the model itself. RAG systems face compounded vulnerabilities: (1) *Retrieval poisoning*—attackers can compromise external knowledge bases or vector stores to inject malicious content that gets retrieved and incorporated into model responses; (2) *Indirect prompt injection via retrieved documents*—as demonstrated in recent work [8], adversaries can embed malicious instructions in documents that are likely to be retrieved, effectively injecting prompts through the retrieval pathway; (3) *Query manipulation*—carefully crafted user queries could exploit the retrieval mechanism to access unauthorized information or trigger retrieval of attacker-controlled content; and (4) *Context window exploitation*—retrieved content consumes context window space, and adversaries could trigger retrieval of large irrelevant documents to perform denial-of-service or displace legitimate system instructions. However, RAG also enables novel defense opportunities, such as real-time retrieval of updated security policies, dynamic threat intelligence integration, and separation of static model knowledge from updateable security contexts.

Research Directions. The security implications of these emerging architectures remain understudied. We advocate for architecture-aware security research that: (1) develops threat models specific to each architectural paradigm; (2) investigates whether architectural innovations inherently mitigate or exacerbate known vulnerabilities; (3) designs security mechanisms that leverage architectural features (e.g., using MoE routing for threat detection, employing SSM states for anomaly monitoring); and (4) establishes security benchmarks tailored to architectural characteristics. As the field moves beyond pure Transformer models, security analysis must evolve in parallel to ensure that architectural progress does not inadvertently introduce new attack vectors.

4.3 Training Layer: Systemic Flaws in the Learning Process

The training process of large language models, foundational to their capabilities, exhibits systemic flaws across multiple levels, inadvertently providing a basis for prompt injection attacks. These vulnerabilities,

spanning from representation learning to optimization strategies, are inherent limitations of the learning mechanism that accumulate during knowledge acquisition, creating exploitable weaknesses.

4.3.1 *Inherent Biases in Representation Learning*

Large language models' representation learning process introduces systematic biases that become exploitation points for attacks, including gender, race, and professional biases encoded in word embeddings as geometric relationships [64], improper semantic cluster associations placing unrelated concepts close together due to spurious correlations, and frequency bias that marginalizes low-frequency concepts while centralizing common words [70]. Contextual representations create new attack vectors through dynamic adjustment mechanisms, where contextual pollution propagates biased content via attention mechanisms [85] and representation drift in long sequences causes semantic deviation from original meanings [101]. Polysemy resolution bias favors frequent training interpretations when handling ambiguous expressions [97], allowing attackers to construct contexts that force specific, harmful interpretations of otherwise harmless expressions through statistical disambiguation manipulation. Deep neural networks suffer from inconsistent hierarchical representations and lack cross-layer consistency checks, enabling attackers to exploit the separation between shallow and deep understanding as well as non-monotonic representation evolution to manipulate model behavior [78,86,87].

4.3.2 *Convergence Bias in the Optimization Process*

Current optimization methods for large language models contain systemic flaws that create exploitable vulnerabilities, including multimodal loss landscapes causing convergence to suboptimal solutions with predictable behavioral patterns [77], imbalanced gradient impacts leading to uneven capability development [62], and optimization path randomness producing models with varying vulnerabilities across different training runs [98]. Mini-batch stochastic gradient descent introduces accumulated biases through uneven batch sampling that causes differential fitting for various data types [103] and noisy gradient estimation that leads to unstable behavioral patterns [72]. Batch normalization creates training-inference inconsistencies by using different statistics in each phase [63], allowing attackers to exploit these optimization flaws to identify model weaknesses and construct targeted attacks that trigger abnormal behaviors or bypass safety mechanisms.

4.3.3 *Conflicts and Interference in Multi-Task Learning*

Modern large language models' multi-task learning capabilities introduce inter-task conflicts that attackers can exploit, particularly through the trade-off conflict between accuracy and safety where improving task-specific performance may compromise safety requirements [88], the difficulty in balancing generality and specialty that leads to inconsistent behavior in specialized domains, and the optimization conflict between efficiency and quality where computational compromises create attack opportunities [89]. Multi-task processing requires rapid task switching that introduces cognitive load and potential errors, creating vulnerabilities through incomplete state retention during transitions that can lead to information contamination across tasks [71]. The cognitive resource consumption of frequent task switching can degrade model performance and reduce safety checking capabilities [60], allowing attackers to construct complex inputs requiring multiple task switches to exhaust computational resources. Attackers can exploit these multi-task vulnerabilities by disguising malicious requests as legitimate task requirements, leveraging specialized domain knowledge gaps, forcing simplified processing through resource-intensive inputs, and implementing cross-task state contamination attacks where malicious information planted in one task activates in subsequent tasks.

4.4 Trends and Challenges

4.4.1 Development Trends

Research on large language model vulnerabilities to prompt injection attacks is evolving across three interconnected levels. At the philosophical level, the field is shifting from seeking perfect value alignment solutions to accepting prompt injection as rooted in fundamental dilemmas about moral relativism and cultural diversity, focusing on frameworks that navigate inherent value conflicts [90]. At the technical level, studies are transitioning from reactive post-hoc filtering to proactive architectural redesign that addresses inherent Transformer vulnerabilities, emphasizing attention mechanisms with built-in security properties [28]. At the training level, research is moving toward holistic paradigms that integrate security considerations throughout the entire learning pipeline rather than treating safety as an add-on component, embedding security awareness into pre-training, fine-tuning, and reinforcement learning processes [102].

4.4.2 Fundamental Challenges

Despite technological advancements, prompt injection attacks persist due to the inherent trade-off between enhancing instruction following and increasing vulnerability to malicious inputs. This fundamental contradiction highlights the challenge of designing models with strong general capabilities that resist harmful instructions, while navigating the ethical dilemma of value alignment across diverse global contexts, where “security” and “harmful” are culturally defined. A future challenge is to create a security framework that flexibly adapts to different cultural backgrounds and legal systems while upholding basic human ethical bottom lines [84,102].

5 RQ3: What Defense Mechanisms Have Been Developed to Mitigate Prompt Injection Attacks?

To answer our research question, we analyzed defense mechanism studies published between 2022 and 2025. Based on this literature, we categorize prompt injection defenses into three main types: **Input Preprocessing and Filtering**, **System Architecture Defenses**, and **Model-Level Defenses**. Table 6 summarizes 37 representative defense studies.

Table 6: Summary of prompt injection attack defenses

Index	Year	Defense category	Authors	Core defense strategy
1	2023	Input preprocessing	Jain et al. [15]	Baseline Defense: Paraphrasing reconstruction to disrupt adversarial suffix attacks
2	2023	Input preprocessing	Kumar et al. [117]	Erase-and-check framework: Provable security guarantees
3	2023	Input preprocessing	Madaan et al. [118]	Self-Refine: Iterative output optimization through self-reflection
4	2023	Input preprocessing	Robey et al. [16]	SmoothLLM: Random perturbation and prediction aggregation detection
5	2023	System architecture	Mialon et al. [31]	Sandbox isolation: Restricting LLM external system access
6	2023	System architecture	Rebedea et al. [119]	NeMo Guardrails: Multi-layer verification mechanism

(Continued)

Table 6 (continued)

Index	Year	Defense category	Authors	Core defense strategy
7	2023	Model defense	Wang et al. [17]	SELF-GUARD: Safety training and self-censorship
8	2023	Model defense	Choi et al. [120]	Parameterization of fixed inputs (prompts) into model parameters to decouple them from inference-time inputs
9	2024	Input preprocessing	Zhang et al. [121]	Goal priority defense: Inserting defensive tokens
10	2024	Input preprocessing	Chen et al. [122]	Fight fire with fire: Converting attack techniques into defense tools
11	2024	Input preprocessing	Hines et al. [123]	Spotlighting: Input provenance-based defense
12	2024	Input preprocessing	Khomsky et al. [124]	Multi-layer defense: Prompt + Python + LLM filters
13	2024	Input preprocessing	Phute et al. [125]	LLM self-defense: Zero-shot self-checking mechanism
14	2024	Input preprocessing	Rai et al. [126]	GUARDIAN: Three-tier defense architecture design
15	2024	Input preprocessing	Wang et al. [127]	FATH: Hash authentication-based test-time defense
16	2024	System architecture	Chen et al. [128]	StruQ: Control and data separation structured query
17	2024	System architecture	Debenedetti et al. [37]	AgentDojo: Dynamic security assessment for LLM agents
18	2024	System architecture	Jia et al. [129]	Task Shield: Task-aligned defense against indirect injection
19	2024	System architecture	Pasquini et al. [130]	Mantis: Converting prompt injection into counterattack
20	2024	System architecture	Suo [131]	Signed-Prompt: Instruction signature-based defense
21	2024	System architecture	Wu et al. [132]	F-Secure LLM: Information flow control system-level defense
22	2024	Model defense	Chen et al. [133]	SecAlign: Preference optimization-based defense
23	2024	Model defense	Jacob et al. [134]	PromptShield: Practical deployment detection benchmark
24	2024	Model defense	Mazeika et al. [18]	HarmBench+R2D2: Standardized evaluation and training
25	2024	Model defense	Panterino et al. [135]	Dynamic moving target: Parameter perturbation attack mitigation

(Continued)

Table 6 (continued)

Index	Year	Defense category	Authors	Core defense strategy
26	2024	Model defense	Piet et al. [136]	Jatmo: Task-specific fine-tuning defense method
27	2025	Input preprocessing	Chen et al. [137]	Instruction reference defense: Explicit reference execution instructions
28	2025	Input preprocessing	Lin et al. [138]	UniGuardian: Unified multi-threat detection
29	2025	Input preprocessing	Shi et al. [139]	PromptArmor: Simple and efficient guardrail LLM defense
30	2025	Input preprocessing	Yi et al. [140]	BIPIA: Boundary-aware defense and benchmarking
31	2025	System architecture	Chen et al. [133]	DefensiveToken: Innovative test-time defense solution
32	2025	System architecture	Debenedetti et al. [19]	CaMeL: Software security design principle defense system
33	2025	System architecture	Jia et al. [20]	Critical evaluation: Principled defense evaluation standards
34	2025	System architecture	Zhong et al. [141]	RTBAS: Information flow control tool agent defense
35	2025	Model defense	Li & Liu [142]	InjecGuard: Addressing guardrail over-defense issues
36	2025	Model Defense	Li et al. [143]	PIGuard: MOF training strategy guardrail model
37	2025	Model defense	Liu et al. [144]	DataSentinel: Game theory-based detection method

5.1 Input Preprocessing and Filtering

Perturbation and Filtering Defenses. Several methods defend against prompt injection through input perturbation and output filtering. Robey et al. [16] proposed SmoothLLM, which exploits adversarial suffixes' vulnerability to character-level perturbations. Jain et al. [15] reduced attack success rates through paraphrasing, while Kumar et al. [117] introduced the erase-and-check framework, providing the first provably robust defense by systematically deleting and checking token subsequences. Classifier-based approaches can capture complex attacks but require substantial labeled data [15,117].

Self-Evaluation and Instruction-Aware Defenses. Recent approaches leverage LLMs' intrinsic capabilities for defense. Madaan et al. [118] proposed Self-Refine, enabling iterative self-evaluation and output optimization through feedback mechanisms. Chen et al. [137] shifted the paradigm from blocking malicious instructions to identifying and filtering responses based on explicit instruction referencing. Phute et al. [125] introduced LLM SELF DEFENSE, utilizing models' inherent ability to detect harmful content without additional training. Zhang et al. [121] proposed target prioritization through special control instructions during inference and training phases.

Attack-as-Defense and Source Distinction. Innovative defense paradigms exploit attack mechanisms themselves. Chen et al. [122] proposed an "attack-as-defense" paradigm that leverages attack structure and

intent for defense. Yi et al. [140] developed the BIPIA benchmark with a dual-defense mechanism based on attack analysis. Hines et al. [123] introduced SpotLighting, enhancing LLMs' ability to distinguish between system instructions and external data through prompt engineering.

Multi-Layer Defense Systems. Comprehensive defense architectures employ multiple protection layers. Khomsky et al. [124] analyzed multi-layer systems consisting of defense prompts, Python filters, and LLM filters in the SaTML 2024 CTF competition. Rai et al. [126] proposed GUARDIAN, a three-layer defense framework. Wang et al. [127] introduced FATH, transforming defense into a cryptographic authentication problem using HMAC-based mechanisms. Shi et al. [139] developed PromptArmor, converting off-the-shelf LLMs into security guardrails through meticulously designed prompts.

Unified and Domain-Specific Defenses. Recent work addresses multiple attack types and specialized scenarios. Lin et al. [138] proposed UniGuardian, the first unified defense against prompt injection, backdoor, and adversarial attacks by defining them as Prompt Triggered Attacks (PTA). Zhang et al. [145] introduced Mixed Encoding (MoE-Defense) using multiple character encodings like Base64. Sharma et al. [146] developed a two-stage framework for image-based attacks on multimodal LLMs. Wen et al. [147] proposed instruction detection-based defense for RAG systems by analyzing internal behavioral state changes. Salem et al. [148] introduced Maatphor for automated generation and evaluation of attack variants. Hung et al. [149] discovered the “distraction effect” through attention pattern analysis.

5.2 Model-Level Defenses

SELF-GUARD, proposed by Wang et al. [17], leverages a two-stage training approach, enabling large language models to self-censor and flag harmful responses. Mazeika et al. developed HarmBench, a standardized evaluation framework [18], and proposed R2D2, an adversarial training method that uses “away-from-loss” and “towards-loss” to train models to refuse to output harmful content. By parametrically injecting fixed prompts into the model, Choi et al.'s Prompt Injection (PI) method shifts fixed instructions from the input layer to the parameter layer via fine-tuning or distillation, aiming to reduce inference latency and computational costs for long prompts [120]. Piet et al. [136] proposed the Jatmo method to defend against prompt injection attacks. Its core principle lies in task-specific fine-tuning, transforming a general instruction-following model into a specialized model for a single task. Panterino et al. [135] pioneered a dynamic Moving Target Defense mechanism that continuously alters the parameters and configurations of large language models to create an unpredictable attack environment, thereby effectively defending against prompt injection attacks. Liu et al. [40] proposed an automatic prompt injection attack based on gradient optimization. SecAlign, proposed by Chen et al. [133], constructs preference datasets of safe and unsafe responses and employs preference optimization techniques. Li and Liu [142], through the InjecGuard model and MOF training strategy, systematically address the over-defense problem of existing Prompt guard models, specifically manifesting as trigger word bias and shortcut learning leading to misclassification of benign inputs. PIGuard [143] innovatively introduces the Mitigating Over-defense for Free training strategy, aiming to solve the over-defense problem of existing Prompt guard models, and quantifies and resolves the model's bias towards common trigger words by constructing the NotInject evaluation dataset. Jacob et al. [134] proposed PromptShield, a comprehensive benchmark for training and evaluating Prompt injection detectors, providing an important tool for building deployable Prompt injection defense systems. DataSentinel, proposed by Liu et al. [144], is a game-theoretic prompt injection attack detection method whose counterintuitive strategy is to enhance detection capabilities by fine-tuning to increase the vulnerability of the detection LLM.

5.3 System Architecture Defense

Agent Security and Tool-Based Defenses. Several frameworks address security challenges in LLM agent systems. DeBenedetti et al. [37] proposed AgentDojo, the first dynamic security evaluation framework for LLM agents in untrusted data environments. Mialon et al. [31] emphasized establishing security boundaries when LLMs call external tools in their Augmented Language Models survey. Rebedea et al. [119] introduced NeMo Guardrails, a multi-layered validation tool with dual input-output auditing. Zhong et al. [141] proposed RTBAS, applying information flow control to Tool-Based Agent Systems. Jia et al. [129] introduced Task Shield, shifting security focus from harmfulness detection to task consistency verification.

Architectural and System-Level Defenses. Fundamental architectural approaches address prompt injection at the system design level. Chen et al. [128] proposed StruQ, treating prompt injection as traditional injection attacks and applying “separation of control and data” principles at the API level. Wu et al. [132] introduced the f-secure LLM system based on Information Flow Control (IFC), separating planner and executor functions. Suo et al. [131] proposed Signed-Prompt, establishing trust verification through digital signatures for sensitive instructions. DeBenedetti et al. [19] introduced the CaMeL framework, building protective system layers around LLMs inspired by traditional software security.

Embedding-Based and Token-Level Defenses. Methods operating at the embedding and token levels provide fine-grained protection. Chen et al. [133] proposed DefensiveToken, optimizing embedding vectors of special defense tokens without altering model parameters. Ayub et al. [150] developed detection methods based on distributional differences between malicious and benign prompts in embedding space. Rahman et al. [151] leveraged multilingual BERT models for embedded representations to detect malicious attacks.

Detection and Removal Frameworks. Comprehensive frameworks combine detection with post-processing mechanisms. Chen et al. [152] proposed a complete defense framework including detection and removal stages for indirect attacks. Li et al. [153] introduced GenTel-Safe with the GenTel-Shield detection method and GenTel-Bench evaluation benchmark. Pasquini et al. [130] proposed Mantis, transforming prompt injection from a vulnerability into a strategic asset. Rossi et al. [42] developed hierarchical defense strategies through systematic attack classification.

Domain-Specific and Specialized Defenses. Targeted approaches address specific application scenarios and attack vectors. Sharma et al. [154] proposed SPML, the first domain-specific language for defending chatbot prompt attacks. Pedro et al. [155] systematically studied Prompt-to-SQL injection attacks in the Langchain framework. Sharma et al. [146] developed two-stage frameworks for image-based attacks on multimodal LLMs. Lee et al. [156] proposed human-AI collaborative frameworks for military LLMs in federated learning environments.

Evaluation, Testing, and Multi-Dimensional Analysis. Critical evaluation and comprehensive analysis frameworks advance defense understanding. Jia et al. [20] proposed principled defense evaluation frameworks emphasizing diverse attack testing. Zhan et al. [157] conducted adaptive attack evaluations revealing fundamental vulnerabilities in current defenses. Yu et al. [158] introduced PROMPTFUZZ, a black-box fuzzing framework applying software testing techniques. Kumar et al. [24] proposed multi-dimensional attack space analysis guiding hierarchical defense architectures. Zhang et al. [145] identified various strategies through a systematic review of 36 papers (2022–2025).

Integrated and Iterative Defense Approaches. Holistic methods combine multiple defense mechanisms and adaptive strategies. Deng et al. [159] proposed integrated offense-defense frameworks with iterative adversarial fine-tuning. Mudarova et al. [160] developed comprehensive ML-based protection solutions with complete algorithm-implementation-verification processes. Liang et al. [49] constructed multi-layered prompt protection frameworks using obfuscation, attention interference, and output filtering.

Rehberger [4] highlighted fundamental challenges stemming from rapid attack innovation and limited mitigation effectiveness.

5.4 Trends and Challenges

Through a systematic analysis of existing prompt injection attack defense mechanisms, as shown in Table 7, we have identified several significant development trends and core challenges in this field.

Table 7: Quantitative comparison of defense mechanism effectiveness against prompt injection attacks

Defense mechanism	Authors	Benchmark/ Dataset	Baseline ASR	ASR with Defense
SmoothLLM (<i>Vicuna for GCG</i>)	Robey et al. [16]	AdvBench	98.1%	0.8%
Paraphrasing (<i>Vicuna</i>)	Jain et al. [15]	Adversarial prompts	79.0%	5.0%
Erase-and-Check (<i>DistilBERT</i>)	Kumar et al. [117]	Adversarial prompts	100%	0%
NeMo Guardrails	Rebedea et al. [119]	Anthropic red-teaming dataset	7.0%	1.0%
Spotlighting (<i>GPT-3.5-Turbo</i>)	Hines et al. [123]	Synthetic dataset (Document Summarization)	50.0%	3.1%
Prompt Sandwiching (<i>GPT-4o</i>)	Debenedetti et al. [37]	AgentDojo (Security Tasks)	57.69%	27.82%
Boundary-aware (<i>GPT-4</i>)	Yi et al. [140]	BIPIA Benchmark	31.03%	24.08%

5.4.1 Limitations and Optimization Directions for Current Defenses

While the defense strategies discussed above provide valuable protection against prompt injection attacks, they face significant limitations in real-world deployment. **Input filtering mechanisms** suffer from high false positive rates (15%–30%) and can be easily evaded through simple obfuscation techniques such as encoding or character substitution, with detection rates dropping from over 90% for known attacks to below 40% for novel variants. **Prompt engineering approaches**, despite their simplicity, lack formal security guarantees and can be bypassed by sophisticated context-ignoring attacks with 40%–60% success rates. **Model fine-tuning methods** like RLHF require prohibitive computational resources and show limited generalization to unseen attack patterns. **Architectural defenses** such as dual-LLM systems introduce substantial latency (200–500 ms per query) and operational costs. To address these limitations, we recommend: (1) adopting *multi-layered defense-in-depth strategies* that combine lightweight rule-based pre-filtering with adaptive ML-based detection; (2) developing *context-aware and dynamic defense mechanisms* that adjust security strictness based on real-time threat assessment; (3) establishing *continuous learning pipelines* that update defense models using adversarial training with newly discovered attack patterns; and (4) creating *standardized benchmarks and evaluation frameworks* to systematically assess defense effectiveness across diverse attack scenarios and model architectures. Future research should prioritize developing defenses with formal security guarantees while maintaining acceptable performance overhead and user experience.

5.4.2 Development Trends

Multi-layered Evolution of Defense Strategies Defense against large language models has evolved from single-point solutions to multi-layered strategies. Early methods focused on isolated filtering. However, increasingly sophisticated attacks have necessitated complex, multi-tiered architectures to enhance overall defense effectiveness.

Intelligent and Adaptive Defense Mechanisms Defense technology is evolving from static rules to intelligent, adaptive mechanisms, exemplified by methods. Panterino et al.'s dynamic moving target defense [135] introduces continuous parameter changes to create unpredictable defense environments for LLMs.

Security by Design at the System Architecture Level The defense shift from post-hoc remediation to Security by Design is evident in recent architectural re-engineering efforts. Similarly, the CaMeL framework [19] establishes a protective system layer around LLMs, reflecting a trend towards systematic security design for robust guarantees.

5.4.3 Core Challenges

Fundamental Dilemma of Offensive-Defensive Asymmetry Attackers possess a significant advantage in resources and time, allowing them to optimize sophisticated attack strategies while defenders often operate under real-time constraints with limited resources. This fundamental asymmetry, particularly evident in LLM prompt injection, places defenders in a perpetual reactive state due to rapid attack innovations and the limited effectiveness of current mitigation strategies.

Protection Gaps for Multimodal Attacks The emergence of multimodal injection attacks highlights critical vulnerabilities in cross-modal threat protection, as traditional defenses struggle with malicious instructions embedded in non-text modalities.

Balancing Defense Effectiveness and System Usability Existing defense mechanisms balance security and usability, with current Prompt guardrail models showing over-defense through trigger word bias and shortcut learning that misclassify benign inputs. A key challenge is to achieve robust security without compromising normal functionality.

5.4.4 Efficiency and Latency Trade-Offs

A critical consideration for real-world deployment, as highlighted by our analysis, is the computational overhead and latency introduced by various defense mechanisms. These costs create a complex trade-off between security robustness and system efficiency, with implications differing drastically across defense architectures.

System-Level and Pre-Processing Overheads: Some architectures, while effective, introduce significant, fixed latency. For instance, the **NeMo Guardrails** framework utilizes a multi-layered validation system involving a “three-step CoT prompting approach” [119]. This process inherently incurs substantial overhead, estimated to be about **3 times the latency and cost** of a standard, undefended LLM call [119]. Similarly, input pre-processing defenses like **Paraphrasing** require an external call to another powerful LLM (e.g., GPT-3.5-turbo) simply to sanitize the input, effectively doubling the inference latency or more before the primary model even begins its task [15].

Scalable Latency in Perturbation Defenses: Perturbation-based defenses like **SmoothLLM** introduce a scalable overhead. The defense's cost is directly proportional to the number of samples (N), requiring “ N times more queries relative to an undefended LLM” [16]. This creates a clear security-efficiency trade-off:

higher N provides greater robustness but increases cost. For example, running $N = 10$ samples (a $10\times$ query increase) took approximately 4.2–4.4 s on an A100/A6000 GPU [16]. However, even one extra query ($N = 2$) demonstrated a $5.7\times$ to $18.6\times$ reduction in ASR on Llama2, highlighting a significant security gain for a $2\times$ latency cost [16].

Filter Model and Complexity Costs: The **Erase-and-Check** framework provides the clearest example of how cost is influenced by both the filter model and the attack complexity [117].

- **Filter Model:** Using a large model (Llama 2) as the safety filter is slow (approx. 1.1 s for suffix mode, $d = 10$), whereas using a fine-tuned, smaller classifier (DistilBERT) is “significantly faster,” achieving the same task in under 0.1 s [117].
- **Attack Complexity:** The cost escalates rapidly when certifying against more complex attacks, even with the fast DistilBERT filter. While suffix mode ($d = 10$) is fast (approx. 0.04 s), insertion mode ($d = 12$) takes approx. 0.1 s, and the more complex infusion mode ($d = 3$) takes over 1 s, with the Llama 2 filter taking a prohibitive **61 s** for the same task [117].

This demonstrates that the computational cost of defense ranges from negligible (e.g., a lightweight classifier for simple attacks) to prohibitive (e.g., using an LLM to certify against complex infusion attacks), making this trade-off a central challenge for practical deployment.

5.4.5 The Co-Evolutionary Arms Race: A Temporal Analysis

The offensive-defensive asymmetry mentioned in Section 5.4.3 is not static; rather, it is a dynamic co-evolutionary arms race. Our systematic review of the 2022–2025 literature reveals a clear pattern of attacks and defenses evolving in direct response to one another. This temporal progression is illustrated in Table 8. The timeline begins in 2022 with foundational **Direct Injection** attacks [7], which were quickly met by simple **Input Preprocessing** defenses like paraphrasing [15] and perturbation [16] in 2023. Attackers immediately adapted; optimization-driven attacks like **GCG** [56] emerged specifically to generate adversarial suffixes that are robust to such simple filtering. This, in turn, spurred the development of more robust **Model-Level Defenses**, such as adversarial training [18]. As defenses for direct text injection hardened, attackers pivoted to entirely new vectors, including **Multimodal Attacks** [13,14] and **Agent-Based** indirect attacks [37]. This forced defenses to evolve once more, moving towards **System-Level Architectures** like StruQ [128] and CaMeL [19] that focus on fundamental “Security by Design” rather than reactive filtering. Each defensive layer is thus met with a new, more sophisticated attack vector, perfectly illustrating the co-evolutionary cycle.

Table 8: Timeline of the attack-defense co-evolutionary arms race (2022–2025). This table illustrates how attack sophistication (left column) has increased in direct response to new defense mechanisms (right column)

Year	Attack evolution (Milestone & Threat)	Defense response (Milestone & Mitigation)
2022	Initial Attack: Direct Injection Foundational instruction override attacks (e.g., “Ignore previous instructions”) are proposed [7].	Baseline: Implicit Defenses Model behavior is controlled only by basic system prompts and initial alignment training.
	Vector Pivot: Indirect Injection Attacks move to data sources (RAG, documents, webpages) to bypass direct input filtering [8].	Layer 1: Input Preprocessing Defenses react by filtering the user’s prompt (e.g., Paraphrasing [15], Perturbation/SmoothLLM [16]).

(Continued)

Table 8 (continued)

Year	Attack evolution (Milestone & Threat)	Defense response (Milestone & Mitigation)
Late 2023	Evasion: Optimization-Driven Attacks Attacks (e.g., GCG) use gradient-based optimization specifically to find “adversarial suffixes” that survive perturbation and filtering [36,56].	Layer 2: Model-Level Defenses Defenses respond by hardening the model itself (e.g., Adversarial Training/HarmBench [18], Self-Guard [17]).
2024	Vector Pivot: New Modalities Attacks shift to entirely new, undefended surfaces: Multimodal (images, functions) [12–14] and Agent-Based (tools) [37].	Layer 3: System-Level Architecture Defenses move beyond the prompt to redesign the system (e.g., StruQ [128], f-secure LLM [132], Signed-Prompt [131]).
2025	Stealth Attacks Attacks become more complex, targeting data flows [48] or physical environments (EnvInjection [50]).	Layer 4: Security by Design Defensive frameworks mature to focus on holistic, provable security principles (e.g., CaMeL [19], Critical Evaluation [20]).

5.5 Practical Deployment Guidelines

While our analysis has examined individual defense mechanisms and their theoretical properties, practitioners face the challenge of translating these insights into concrete deployment decisions. We synthesize our findings into a practical decision framework that guides defense selection based on three critical dimensions: *application requirements*, *threat models*, and *resource constraints*.

Application-Specific Recommendations. The choice of defense mechanisms should be primarily driven by application context. For *high-security applications* (e.g., financial services, healthcare, legal advisory systems), we recommend a defense-in-depth approach combining: (1) input-based filtering with strict whitelisting and semantic analysis to catch malicious prompts at entry; (2) model-based defenses such as adversarial training or safety-tuned models to ensure robust internal processing; and (3) output-based validation with LLM-as-a-judge mechanisms to provide a final safety check. This multi-layer strategy, while incurring higher computational overhead (typically 2–3× latency increase), achieves ASR < 5% as discussed in Section 6.3. For *general-purpose assistants* with moderate security requirements, a balanced approach combining lightweight input filtering with instruction-hierarchy enforcement offers practical security (ASR < 15%) with acceptable overhead (<50% latency increase). For *resource-constrained environments* (e.g., edge deployments, mobile applications), prioritize efficient input-based defenses such as prompt-based detection or lightweight classifiers that can run locally, accepting slightly higher ASR in exchange for a minimal computational footprint. For *agent-based systems* interacting with external tools and data sources, indirect prompt injection poses the primary threat; deploy specialized defenses, including data-prompt isolation techniques, retrieval content sanitization, and privilege separation as discussed in recent work on agent security.

Threat-Model-Driven Selection. Defense effectiveness varies significantly across attack types, as evidenced in Table 9. Against *direct jailbreak attempts* (e.g., role-playing, hypothetical scenarios), input-based semantic analysis and model-based safety tuning prove most effective, as these attacks rely on exploiting

instruction-following behavior. Against *indirect prompt injections* through external data sources, defenses must focus on data provenance tracking and content isolation, as traditional input filtering cannot distinguish between legitimate user inputs and injected malicious instructions. For *multi-turn attacks* that gradually escalate malicious intent across conversation history, stateful monitoring mechanisms and conversation-level anomaly detection become essential, as single-turn defenses may miss the attack pattern. Against *adversarial perturbations* and token-level manipulations, model-based defenses with adversarial training or input preprocessing (e.g., paraphrasing, retokenization) offer better resilience than semantic-level analysis. Practitioners should conduct threat modeling exercises to identify their primary attack vectors and prioritize defenses accordingly.

Table 9: Summary of prompt injection security evaluation platforms (2023–2025). Columns represent: **Index** (sequential numbering), **Year** (publication timeline), **Authors** (platform developers), **Attack Methods** (evaluated attack techniques including position-based, obfuscation, social engineering, and indirect injection), **Defense Strategies** (assessed mitigation approaches such as system prompt hardening, delimiters, and detection methods), **Dataset** (benchmark datasets from repurposed NLP corpora to specialized platforms like Tensor Trust and AgentDojo), **Attack Metrics** (quantitative effectiveness measures with ASR as the emerging standard), and **Defense Metrics** (defense effectiveness measures through comparative analysis and utility preservation)

Index	Year	Authors	Attack methods	Defense strategies	Dataset	Attack metrics	Defense metrics
1	2023	Li et al. [161]	Position-based, Instruction type, Specific phrase	System prompt, Context order	Natural Questions, TriviaQA, SQuAD, HotpotQA	PDR, IDR	PDR, IDR comparison
2	2023	Toyer et al. [35]	Game-based, LDA topic modeling, Adversarial tokens	Community-driven, Delimiters, Instruction repetition	Tensor Trust	HRR, ERR	HRR, ERR comparison
3	2023	Yip et al. [162]	Payload splitting, Obfuscation, Role play, Indirect injection	Not specified	A self-built dataset of 115 attacks	RR, AIM, ASP et al.	WRS comparison
4	2024	Sang et al. [163]	Character-level, Word-level, Sentence-level, Semantic-level	Not specified	Microsoft PromptBench	Response Accuracy, Adversarial Robustness, FPR et al.	Not specified
5	2024	Liu et al. [28]	Naive attack, escape characters, context ignoring, fake completion, combined attack	Prevention-based, Detection-based	MRPC, Jfleg, HSOL, RTE, SST2, SMS Spam, Gigaword	ASV, MR, FPR et al.	ASV, MR, PNA-T et al.
6	2024	Debenedetti et al. [37]	“Important message”, “ignore previous instructions”	Data Delimiters, PI Detection, Prompt Sandwiching, Tool Filter	AgentDojo	ASR	ASR, Benign Utility comparison
7	2024	Kumar et al. [24]	Direct attacks, indirect attacks, roleplay, payload splitting, obfuscation	System prompt hardening, Prompt perturbation, Tool limitation	No specific dataset	Not specified	Not specified

(Continued)

Table 9 (continued)

Index	Year	Authors	Attack methods	Defense strategies	Dataset	Attack metrics	Defense metrics
8	2024	Ramakrishna et al. [164]	Execute unauthorized actions, access sensitive information, distract model	Prompt-based mitigation strategy	SQuAD + 150 generated APIs	Attack success rate	Attack Success Rate comparison
9	2024	Zhan et al. [2]	Direct harm attacks, data stealing attacks, “hacking prompt”	Fine-tuned method comparison	INJECAGENT	ASR, ASR-valid, sensitivity rate	ASR comparison
10	2025	Evtimov et al. [165]	Plain-text injection, URL injection	Instruction hierarchy, Defensive system prompt	WASP benchmark	ASR-intermediate, ASR-end-to-end, Utility	ASR-intermediate, ASR-end-to-end comparison

Defense Layering and Implementation Priorities. Our analysis throughout [Section 5](#) demonstrates that no single defense mechanism provides comprehensive protection. We recommend a phased deployment approach: *Phase 1 (Immediate)*-Deploy lightweight input-based filtering as a first line of defense, providing immediate risk reduction with minimal integration complexity; *Phase 2 (Short-term)*-Implement output-based validation or LLM-as-a-judge mechanisms to catch attacks that bypass input filters, creating a two-layer defense with complementary coverage; *Phase 3 (Long-term)*-Invest in model-based defenses through fine-tuning, adversarial training, or safety-aligned model selection, which offer more fundamental security improvements but require significant resources and expertise. When combining defenses, prioritize complementary mechanisms that address different attack stages: input filtering catches obvious attacks early, model-based defenses provide robust internal processing, and output validation serves as a final safety net. Monitor false positive rates carefully, as overly aggressive defense combinations can severely degrade user experience—our recommended target is FPR <2% on legitimate queries. Finally, implement continuous monitoring and adaptive defense strategies, as attackers constantly evolve their techniques; regularly evaluate defense effectiveness against emerging attack patterns and update defense parameters accordingly.

This framework provides a systematic methodology for practitioners to navigate the complex landscape of prompt injection defenses, balancing security effectiveness, operational efficiency, and usability preservation based on their specific deployment context.

6 RQ4: What Datasets and Evaluation Metrics Have Been Established for Prompt Injection Attack Research?

To systematically address this question, we summarize existing research and compile multiple datasets and evaluation metrics for prompt injection attacks against Large Language Models in [Table 9](#). This table details the year, author, attack, and defense methods, dataset, and evaluation metrics for each platform. Following this, we briefly introduce each evaluation framework, culminating in a discussion of Trends and Challenges.

6.1 Evaluation Platform

Li et al. [161] proposed a benchmark framework for evaluating the instruction-following robustness of large language models by simulating real-world dialogue system scenarios where models must answer user queries based on web search results that may contain injected instructions. The research focuses on “benign adversarial instructions” rather than malicious output-oriented attacks, evaluating how models handle priority conflicts between original and injected instructions through position-based attacks, different instruction types, and specific injection phrases like “ignore previous prompt.” The study introduced two core metrics: Performance Drop Rate (PDR) defined as $PDR(f) = \frac{Acc(f) - Adv(f)}{Acc(f)}$ to measure the accuracy decrease after instruction injection, and Instruction Discrimination Rate (IDR) is defined as $IDR(f) = \frac{Adv(f)}{Adv(f) + Adv'(f)}$ to assess the model’s tendency to prioritize original vs. injected instructions. Basic defense strategies were evaluated, including explicit system prompts instructing models to ignore instructions within XML tags and changing the order of user questions and context, with effectiveness measured by comparing PDR and IDR performance with and without defense mechanisms. The evaluation utilized four representative question-answering datasets: NaturalQuestions for real user queries, TriviaQA for question-answer pairs with supporting documents, SQuAD for Wikipedia-based reading comprehension, and HotpotQA for multi-hop reasoning tasks.

Toyer et al. [35] proposed Tensor Trust, an online game that crowdsourced the largest dataset of human-generated adversarial samples against large language models, creating benchmarks to evaluate robustness against prompt extraction and hijacking attacks. The attack methods were derived from player creations and generalized using LDA topic modeling, including confusion phrases like “End ambiguity,” roleplay requests to leak information, and model-specific adversarial tokens such as “artisanlib” to bypass defenses. The study employed three key evaluation metrics: Hijacking Robustness Rate (HRR), measuring avoidance of “access granted” outputs during hijacking attacks, Extraction Robustness Rate (ERR), measuring prevention of verbatim access code outputs during extraction attacks, and Defense Validity (DV), measuring correct “access granted” responses to legitimate access codes. While no specific defense strategies were proposed, the research analyzed successful player defense patterns, including explicit instructions for specific access codes, warnings against following user input instructions, role clarification, delimiter usage, and instruction repetition. The dataset was generated through the online game rather than traditional publicly available datasets, with defense effectiveness measured through HRR and ERR performance under various attack and defense configurations.

Yip et al. [162] proposed a novel three-stage evaluation framework to quantify LLM-integrated application resilience against prompt injection attacks: constructing a dataset of 115 representative attacks, utilizing a second LLM as evaluator to generate quantitative impact scores with explanations, and calculating weighted resilience scores where higher-impact attacks receive greater weights. The study evaluates attack techniques across four categories (“Manipulated Content”, “Fraud”, “Harm and Destruction”, “Misinformation”), including payload splitting, obfuscation, ignoring previous prompts, character roleplay, creative dialogue, privacy leakage, and indirect injection. Four key metrics were employed: Relevance Ratio (RR) defined as $RR = \frac{a}{b}$ measuring attack success probability across multiple LLMs with 20% threshold for dataset inclusion; Average Impact Metric (AIM) defined as $AIM = \frac{\sum_{e=1}^f(I_e)}{f}$ quantifying potential negative impact on a 0–5 scale; Attack Success Probability (ASP) defined as $ASP = \frac{S}{T}$ measuring single attack success rate; and Weighted Resilience Score (WRS) defined as $WRS = 100 \times \left(1 - \frac{\sum_{n=1}^m(AIM_n \times ASP_n)}{\sum_{n=1}^m(AIM_n \times 100\%)}\right)$ providing comprehensive resilience assessment ranging 0–100. The framework focuses on resilience evaluation rather than defense strategies, using a custom dataset of 115 attacks collected through literature review and web searches, primarily targeting Llama2 and ChatGLM models.

Sang et al. [163] evaluated prompt injection safety in Anthropic Claude and Mistral Large models using the Microsoft PromptBench dataset, systematically comparing their performance across various adversarial scenarios, including character-level (Deep WordBug, TextBugger), word-level (Text Fooler, Bert Attack), sentence-level (CheckList, Stress Test), and semantic-level (human-crafted) attacks. The study employed comprehensive evaluation metrics, including Response Accuracy, Adversarial Robustness, False Positive/Negative Rates, Context Preservation Score, Semantic Consistency Index, and Robustness to Variations, to assess model safety and integrity under adversarial conditions. While no specific defense strategies were proposed, the research highlighted the effectiveness of Anthropic Claude's advanced safety mechanisms and identified areas for improvement in Mistral Large's handling of context and semantic manipulation. The evaluation utilized the Microsoft PromptBench dataset, which provides diverse and comprehensive prompts designed to test LLM robustness against adversarial inputs across various models, tasks, datasets, prompt types, and attack vectors.

Ramakrishna et al. [164] introduced LLM-PIRATE, an automated framework designed to benchmark indirect prompt injection attacks by embedding attack strings within retrieved documents in RAG systems. It simulates multi-turn conversations and evaluates attack success rate across three verticals: executing unauthorized actions, accessing sensitive information, and distracting the model. The study assessed both white-box and black-box attacks, finding that prompt-based mitigation strategies were largely ineffective. The benchmark leverages a dataset incorporating 150 automatically generated APIs, which allows for the creation of extensible test sets, mitigating test set contamination.

Zhan et al. [2] introduced INJECAGENT, a pioneering benchmark framework for evaluating tool-integrated LLM agents against indirect prompt injection (IPI) attacks, featuring 1054 realistic test cases across 17 user tools and 62 attacker tools. This framework classifies attacks into "direct harm to users" and "exfiltration of private data," and measures security using Attack Success Rate (ASR) and ASR-valid, which accounts for invalid model outputs. The study further explores the influence of "hacking prompts" on attack efficacy and utilizes a custom dataset derived from 330 tools, generating diverse attack scenarios.

Evtimov et al. [165] introduced WASP, a benchmark to evaluate web agent security against prompt injection by simulating realistic threats where attackers control specific web elements. This benchmark assesses attacks targeting multi-step security breaches rather than single tasks, operating in a sandboxed web environment for safety and reproducibility. It measures attack success via ASR-intermediate and ASR-end-to-end, while evaluating defenses like instruction hierarchy and defensive system prompts.

Liu et al. [28] introduced a pioneering framework for prompt injection, defining attacks as forcing an LLM to execute an "injected task" over its "target task." This framework enabled the formalization of existing attacks and the creation of a novel "Combined Attack," along with a comprehensive quantitative benchmark involving five attack types and ten defense strategies across various LLMs and tasks. The study further established quantitative metrics like Performance under No Attacks (PNA), Attack Success Value (ASV), and Matching Rate (MR) to evaluate attack efficacy and defense robustness, categorizing defenses into prevention-based and detection-based strategies.

In [37], AgentDojo was introduced as a dynamic evaluation framework designed to assess the adversarial robustness of AI agents against prompt injection, focusing on attacks from untrusted data. This framework allows researchers to define and evaluate new agent tasks, defenses, and adaptive attacks, simulating real-world agent execution in a stateful, adversarial setting. The study assessed attack methods, primarily a "Important message" attack, and evaluated defenses like Data Delimiters and Prompt Sandwiching by measuring metrics such as Benign Utility and Targeted Attack Success Rate (ASR) across a dataset of 629 security test cases in four environments.

6.2 Trends and Challenges

6.2.1 Development Trends

From the development trajectory of evaluation platforms summarized in [Table 9](#), the research on prompt injection attacks demonstrates the following significant trends:

Shift from static evaluation to dynamic interactive evaluation. Early evaluation frameworks primarily focused on static question-answering. However, recent research constructed dynamic, stateful evaluation environments, reflecting a deeper understanding of real-world application complexities.

Evolution of defense strategy evaluation from passive detection to active prevention. Defense evaluation is evolving from merely detecting attacks to proactively preventing them. This shift is highlighted by the categorization of strategies into prevention-based and detection-based types.

Evaluation environments increasingly align with real-world application scenarios. Recent evaluation frameworks prioritize simulating real-world conditions. These frameworks move beyond technical metrics to include practical aspects like user experience and system usability, reflecting a push towards more applied evaluations.

6.2.2 Core Challenges

Despite the rapid development of evaluation frameworks, several challenges remain:

Lack of uniformity in evaluation standards. Different studies employ significantly varied evaluation metrics and datasets, making it difficult to directly compare research results.

Subjectivity and bias in dataset construction. Datasets, often built on subjective judgments or specific applications, can introduce systematic biases that require validation, even for carefully curated samples.

Lack of cross-modal and multilingual evaluation. Current evaluation frameworks primarily target English text modality. Evaluation of prompt injection attacks in multilingual environments and across multiple modalities remains insufficient.

6.3 Toward a Unified Evaluation Framework

The Need for Standardization. As highlighted in our analysis of existing defense mechanisms ([Table 9](#)), the research community faces a critical challenge: the absence of standardized evaluation metrics makes it difficult to fairly compare different approaches and assess their real-world applicability. Different studies employ varying datasets, metrics, and experimental settings, leading to inconsistent and sometimes incomparable results. To address this gap, we propose a unified evaluation framework that can be adopted by researchers to ensure reproducible and comparable assessments of defense mechanisms. Our framework is grounded in three essential dimensions that collectively capture the multifaceted requirements of effective defenses: **security effectiveness**, **operational efficiency**, and **usability preservation**.

Core Evaluation Dimensions. We advocate for a three-tier evaluation approach. First, **security effectiveness** should be measured using Attack Success Rate (ASR)—the percentage of attacks that successfully bypass the defense—with complementary Defense Success Rate ($DSR = 1 - ASR$). Based on our survey of existing work, we suggest target thresholds of $ASR < 5\%$ for high-security applications (e.g., financial services, healthcare) and $ASR < 15\%$ for general-purpose deployments. Second, **operational efficiency** must account for real-world deployment constraints through metrics including inference latency overhead (milliseconds per query), memory footprint increase (MB), and throughput degradation (queries per second) relative to undefended baselines. Third, **usability preservation** should be quantified via false positive rate (FPR) on

legitimate requests and task completion accuracy on standard benchmarks, ensuring that security measures do not unacceptably degrade user experience or model utility.

Recommended Evaluation Protocols. Building upon the datasets and methodologies identified in Table 9, we recommend that future defense evaluations adopt a standardized protocol using established benchmark suites. Specifically, we suggest mandatory evaluation on: (1) **JailbreakBench** [166] for direct prompt injection attacks (100 diverse malicious behaviors); (2) **HarmBench** [18] for standardized red-teaming scenarios across multiple risk categories; and (3) **AgentDojo** [37] for indirect injection attacks targeting LLM agents. For each defense mechanism, researchers should report all three core metrics (ASR/DSR, efficiency overhead, FPR) with statistical confidence intervals, test robustness against adaptive attacks where adversaries have knowledge of the defense, and evaluate cross-dataset generalization. We believe that widespread adoption of this unified framework will accelerate progress in the field by enabling rigorous, reproducible comparisons and identifying truly effective defense strategies that balance security, efficiency, and usability.

7 Conclusion

This systematic literature review provides a comprehensive analysis of prompt injection attacks against large language models, synthesizing 128 peer-reviewed studies published between 2022 and 2025. Through rigorous methodology following Kitchenham's guidelines, we address critical gaps in understanding attack mechanisms, vulnerability root causes, defense strategies, and evaluation frameworks.

7.1 Summary of Contributions

Our research makes four principal contributions to the field of LLM security:

First, we established a systematic attack classification framework that organizes prompt injection techniques across three dimensions: attack vectors (direct, indirect, and multimodal injection), attack objectives (system prompt leakage, behavior hijacking, privilege escalation, and data exfiltration), and technical implementations (manual crafting, automated generation, and optimization-driven methods). This multi-dimensional taxonomy captures the full spectrum of attack techniques and reveals their evolution from simple instruction override to sophisticated cross-modal manipulation, providing researchers and practitioners with a unified conceptual framework for understanding the threat landscape.

Second, we conducted a comprehensive root cause attribution analysis examining vulnerabilities across philosophical, technical, and training dimensions. Our analysis reveals that prompt injection vulnerabilities stem from: (1) fundamental philosophical dilemmas including value system diversification, unverifiability of alignment status, and inherent conflicts between instruction-following and safety constraints; (2) technical architectural flaws in Transformer attention mechanisms, autoregressive generation, and context understanding; and (3) systematic training deficiencies in representation learning, optimization processes, and multi-task learning. This multi-level framework explains why comprehensive defenses remain elusive despite significant research efforts.

Third, we systematically categorized 37 defense mechanisms into three main approaches: input pre-processing and filtering (including perturbation-based, self-evaluation, and multi-layer defenses), system architecture defenses (featuring agent security frameworks, architectural redesigns, and detection systems), and model-level defenses (encompassing adversarial training, preference optimization, and dynamic protection). Our analysis reveals that while input-level defenses achieve 60%–80% detection rates and advanced architectural approaches demonstrate up to 95% protection against known patterns, significant gaps persist against novel attack vectors, highlighting the ongoing arms race between attackers and defenders.

7.2 Key Findings and Significance

Our systematic analysis yields several significant findings with important implications for LLM security:

Attack Evolution and Sophistication. We documented a clear progression from simple direct injections in 2022 to sophisticated multimodal attacks achieving over 90% success rates against unprotected systems by 2024–2025. Notably, more capable models exhibit higher vulnerability to text-based attacks (Pearson correlation coefficient 0.6635, $p < 0.001$), revealing a fundamental trade-off between instruction-following capability and attack susceptibility. This counterintuitive finding challenges the assumption that more advanced models are inherently more secure.

Fundamental Vulnerability Mechanisms. Our attribution analysis reveals that prompt injection vulnerabilities are not merely implementation bugs but stem from deep-seated issues: philosophical contradictions in value alignment (moral relativism, cultural diversity), architectural limitations (attention mechanism manipulability, autoregressive constraints), and training deficiencies (catastrophic forgetting, reward hacking). This multi-level understanding explains why piecemeal defenses fail and points toward the need for fundamental architectural innovations.

Defense Effectiveness and Limitations. While our review identified 37 defense approaches, empirical evidence shows significant limitations: input filtering suffers from 15%–30% false positive rates and can be evaded through simple obfuscation; prompt engineering lacks formal security guarantees with 40%–60% bypass rates; model fine-tuning shows limited generalization to novel attacks; and architectural defenses introduce substantial latency (200–500 ms per query). These findings underscore the asymmetric advantage attackers maintain in the ongoing security arms race.

Evaluation Framework Gaps. Our analysis reveals critical deficiencies in current evaluation methodologies: lack of standardized metrics across studies, subjective bias in dataset construction, insufficient coverage of multimodal and multilingual scenarios, and limited assessment of defense robustness against adaptive attacks. These gaps hinder reproducibility and comparative analysis, impeding progress toward robust solutions.

The significance of these findings extends beyond academic interest. As LLMs become increasingly integrated into critical applications—from healthcare diagnostics to financial services and autonomous systems—prompt injection attacks pose systemic risks with potential for significant harm. Our comprehensive framework provides the foundation for developing next-generation defenses and informs policy discussions on AI safety regulations.

7.3 Practical Implications

Our research offers actionable insights for multiple stakeholders:

For LLM Developers: Adopt defense-in-depth strategies combining lightweight rule-based pre-filtering with adaptive ML-based detection, implement context-aware dynamic defenses adjusting security strictness based on real-time threat assessment, and establish continuous learning pipelines updating defense models through adversarial training with newly discovered attack patterns.

For Application Designers: Implement architectural safeguards including input source isolation, privilege separation between system instructions and user data, and formal verification of security-critical components. Our analysis of system-level defenses (e.g., F-Secure LLM, CaMeL framework) provides concrete design patterns.

For Security Researchers: Prioritize development of defenses with formal security guarantees, establish standardized benchmarks for reproducible evaluation, and investigate cross-modal attack vectors where current protections are weakest.

For Policymakers: Recognize prompt injection as a systemic vulnerability requiring regulatory attention, establish security certification standards for LLM-based systems in critical domains, and mandate transparency in security testing and vulnerability disclosure.

7.4 Future Research Directions

Based on identified gaps and challenges, we propose the following research priorities:

Formal Verification Methods: Develop mathematically rigorous frameworks for proving defense robustness against defined threat models, moving beyond empirical evaluation to provable security guarantees.

Architectural Innovations: Design next-generation LLM architectures with inherent security properties, including explicit instruction-data separation mechanisms, hierarchical attention with built-in privilege levels, and verifiable alignment constraints.

Standardized Evaluation Protocols: Establish community-wide benchmarks with diverse attack scenarios, multilingual coverage, multimodal threats, and adaptive attack evaluation, enabling reproducible and comparable security assessments.

Multimodal Defense Mechanisms: Address critical gaps in cross-modal attack protection through unified frameworks that detect and mitigate threats across text, image, audio, and video modalities.

Human-AI Collaborative Security: Investigate hybrid approaches combining automated defenses with human oversight for high-stakes decisions, balancing security with usability and transparency.

In conclusion, while prompt injection attacks represent a fundamental challenge to LLM security, our systematic analysis provides a comprehensive foundation for understanding and addressing this critical threat. The path forward requires coordinated efforts across research, industry, and policy domains to develop robust, practical, and formally verified defense mechanisms that enable the safe deployment of LLMs in society.

Acknowledgement: Not applicable.

Funding Statement: This work was supported by 2023 Higher Education Scientific Research Planning Project of China Society of Higher Education (No. 23PG0408), 2023 Philosophy and Social Science Research Programs in Jiangsu Province (No. 2023SJSZ0993), Nantong Science and Technology Project (No. JC2023070), Key Project of Jiangsu Province Education Science 14th Five-Year Plan (Grant No. B-b/2024/02/41) and the Open Fund of Advanced Cryptography and System Security Key Laboratory of Sichuan Province (Grant No. SKLACSS-202407). This work is sponsored by the Cultivation of Young and Middle-Aged Academic Leaders in the “Qing Lan Project” of Jiangsu Province and the 2025 Outstanding Teaching Team in the “Qing Lan Project” of Jiangsu Province. This work is supported partially by Nantong Key Laboratory of Artificial Intelligence New Quality Technology, Jiangsu College of Engineering and Technology.

Author Contributions: Tongcheng Geng conceived and designed the whole study, collected and analyzed the data, and wrote the manuscript; Zhiyuan Xu provided expertise in statistical analysis and contributed to manuscript revisions; Yubin Qu provided expertise in statistical analysis and assisted with data interpretation; W. Eric Wong provided expertise in statistical analysis, assisted with data interpretation, and contributed to manuscript revisions. All authors reviewed the results and approved the final version of the manuscript.

Availability of Data and Materials: Not applicable.

Ethics Approval: Not applicable.

Conflicts of Interest: The authors declare no conflicts of interest to report regarding the present study.

References

1. Red ET. ChatGPT chat with code plugin take down [Internet]; 2023 [cited 2025 Jan 27]. Available from: <https://embracethered.com/blog/posts/2023/chatgpt-chat-with-code-plugin-take-down/>.
2. Zhan Q, Liang Z, Ying Z, Kang D. InjecAgent: benchmarking indirect prompt injections in tool-integrated large language model agents. In: Ku LW, Martins A, Srikumar V, editors. Findings of the Association for Computational Linguistics: ACL 2024. Stroudsburg, PA, USA: ACL; 2024. p. 10471–506.
3. Wang H, Zhong R, Wen J, Steinhardt J. Adaptivebackdoor: backdoored language model agents that detect human overseers. In: Proceedings of the ICML 2024 Next Generation of AI Safety Workshop; 2024 Jul 26; Wien, Austria. p. 1–15.
4. Rehberger J. Trust no AI: prompt injection along the CIA security triad. arXiv:2412.06090. 2024.
5. Red ET. ChatGPT macOS App: persistent data exfiltration [Internet]; 2024 [cited 2025 Jan 27]. Available from: <https://embracethered.com/blog/posts/2024/chatgpt-macos-app-persistent-data-exfiltration/>.
6. Hat B. Black Hat Webcast [Internet]; 2025 [cited 2025 Jan 27]. Available from: <https://www.blackhat.com/html/webcast/06102025.html>.
7. Perez F, Ribeiro I. Ignore previous prompt: attack techniques for language models. arXiv:2211.09527. 2022.
8. Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security. New York, NY, USA: ACM; 2023. p. 79–90.
9. Liu Y, Deng G, Li Y, Wang K, Wang Z, Wang X, et al. Prompt injection attack against LLM-integrated applications. arXiv:2306.05499. 2023.
10. Bagdasaryan E, Hsieh TY, Nassi B, Shmatikov V. Abusing images and sounds for indirect instruction injection in multi-modal LLMs. arXiv:2307.10490. 2023.
11. Qi X, Huang K, Panda A, Henderson P, Wang M, Mittal P. Visual adversarial examples jailbreak aligned large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 38; Palo Alto, CA, USA: AAAI Press; 2024. p. 21527–36.
12. Kwon H, Pak W. Text-based prompt injection attack using mathematical functions in modern large language models. Electronics. 2024;13(24):5008. doi:10.3390/electronics13245008.
13. Wang L, Ying Z, Zhang T, Liang S, Hu S, Zhang M, et al. Manipulating multimodal agents via cross-modal prompt injection. In: Proceedings of the 33rd ACM International Conference on Multimedia. MM '25. New York, NY, USA: ACM; 2025. p. 10955–64. doi:10.1145/3746027.3755211.
14. Clusmann J, Ferber D, Wiest IC, Schneider CV, Brinker TJ, Foersch S, et al. Prompt injection attacks on vision language models in oncology. Nat Commun. 2025;16(1):1239. doi:10.1038/s41467-024-55631-x.
15. Jain N, Schwarzschild A, Wen Y, Somepalli G, Kirchenbauer J, Chiang PY, et al. Baseline defenses for adversarial attacks against aligned language models. arXiv:2309.00614. 2023.
16. Robey A, Wong E, Hassani H, Pappas GJ. Smoothllm: defending large language models against jailbreaking attacks. arXiv:2310.03684. 2023.
17. Wang Z, Yang F, Wang L, Zhao P, Wang H, Chen L, et al. Self-guard: empower the LLM to safeguard itself. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Stroudsburg, PA, USA: ACL; 2024. p. 1648–68.
18. Mazeika M, Phan L, Yin X, Zou A, Wang Z, Mu N, et al. HarmBench: a standardized evaluation framework for automated red teaming and robust refusal. In: Proceedings of the 41st International Conference on Machine Learning. ICML'24; 2024 Jul 21–27; Vienna, Austria. p. 35181–224.

19. Debenedetti E, Shumailov I, Fan T, Hayes J, Carlini N, Fabian D, et al. Defeating prompt injections by design. *arXiv:2503.18813*. 2025.
20. Jia Y, Shao Z, Liu Y, Jia J, Song D, Gong NZ. A critical evaluation of defenses against prompt injection attacks. *arXiv:2505.18333*. 2025.
21. Peng B, Chen K, Li M, Feng P, Bi Z, Liu J, et al. Securing large language models: addressing bias, misinformation, and prompt attacks. *arXiv:2409.08087*. 2024.
22. Rababah B, Wu ST, Kwiatkowski M, Leung CK, Akcora CG. SoK: prompt hacking of large language models. In: 2024 IEEE International Conference on Big Data (BigData). Piscataway, NJ, USA: IEEE; 2024. p. 5392–401.
23. Mathew E. Enhancing security in large language models: a comprehensive review of prompt injection attacks and defenses. *TechRxiv*. 2024.
24. Kumar SS, Cummings M, Stimpson A. Strengthening LLM trust boundaries: a survey of prompt injection attacks. In: 2024 IEEE 4th International Conference on Human-Machine Systems (ICHMS). Piscataway, NJ, USA: IEEE; 2024. p. 1–6.
25. Kitchenham B. Procedures for performing systematic reviews. Keele, UK: Keele University; Eversleigh, NSW, Australia: Empirical Software Engineering National ICT Australia Ltd.; 2004. Keele University Report No.: TR/SE-0401; Report No.: 040001IT.1
26. Zhang H, Babar MA, Tell P. Identifying relevant studies in software engineering. *Inform Softw Technol*. 2011;53(6):625–37. doi:10.1016/j.infsof.2010.12.010.
27. Niu F, Li C, Liu K, Xia X, Lo D. When deep learning meets information retrieval-based bug localization: a survey. *ACM Comput Surv*. 2025;57(11):1–41. doi:10.1145/3734217.
28. Liu Y, Jia Y, Geng R, Jia J, Gong NZ. Formalizing and benchmarking prompt injection attacks and defenses. In: 33rd USENIX Security Symposium (USENIX Security 24). Berkeley, CA, USA: USENIX Association; 2024. p. 1831–47.
29. Kitchenham B, Charters S. Guidelines for performing systematic literature reviews in software engineering. Keele, UK: Keele University; Durham, UK: University of Durham; 2007. Report No.: EBSE-2007-01.
30. Croft R, Xie Y, Babar MA. Data preparation for software vulnerability prediction: a systematic literature review. *IEEE Transact Softw Eng*. 2022;49(3):1044–63. doi:10.1109/tse.2022.3171202.
31. Mialon G, Dessì R, Lomeli M, Nalmpantis C, Pasunuru R, Raileanu R, et al. Augmented language models: a survey. *arXiv:2302.07842*. 2023.
32. Wohlin C. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering; 2014 May 13–14; London, UK. p. 1–10.
33. Hall T, Beecham S, Bowes D, Gray D, Counsell S. A systematic literature review on fault prediction performance in software engineering. *IEEE Transact Softw Eng*. 2011;38(6):1276–304. doi:10.1109/tse.2011.103.
34. Shah R, Feuillede-Montixi Q, Pour S, Tagade A, Casper S, Rando J. Scalable and transferable black-box jailbreaks for language models via persona modulation. *arXiv:2311.03348*. 2023.
35. Toyer S, Watkins O, Mendes EA, Svegliato J, Bailey L, Wang T, et al. Tensor trust: interpretable prompt injection attacks from an online game. *arXiv:2311.01011*. 2023.
36. Wei A, Haghtalab N, Steinhardt J. Jailbroken: how does llm safety training fail? *Adv Neural Inform Process Syst*. 2023;36:80079–110.
37. Debenedetti E, Zhang J, Balunovic M, Beurer-Kellner L, Fischer M, Tramèr F. Agentdojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *Adv Neural Inform Process Syst*. 2024;37:82895–920.
38. Hui B, Yuan H, Gong N, Burlina P, Cao Y. Pleak: prompt leaking attacks against large language model applications. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. New York, NY, USA: ACM; 2024. p. 3600–14.
39. Lee D, Tiwari M. Prompt infection: LLM-to-LLM prompt injection within multi-agent systems. *arXiv:2410.07283*. 2024.
40. Liu X, Yu Z, Zhang Y, Zhang N, Xiao C. Automatic and universal prompt injection attacks against large language models. *arXiv:2403.04957*. 2024.

41. Pasquini D, Strohmeier M, Troncoso C. Neural exec: learning (and learning from) execution triggers for prompt injection attacks. In: Proceedings of the 2024 Workshop on Artificial Intelligence and Security. New York, NY, USA: ACM; 2024. p. 89–100.
42. Rossi S, Michel AM, Mukkamala RR, Thatcher JB. An early categorization of prompt injection attacks on large language models. arXiv:2402.00898. 2024.
43. Shi J, Yuan Z, Liu Y, Huang Y, Zhou P, Sun L, et al. Optimization-based prompt injection attack to LLM-as-a-judge. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. New York, NY, USA: ACM; 2024. p. 660–74.
44. Yan J, Yadav V, Li S, Chen L, Tang Z, Wang H, et al. Backdooring instruction-tuned large language models with virtual prompt injection. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Stroudsburg, PA, USA: ACL; 2024. p. 6065–86.
45. Yu J, Wu Y, Shu D, Jin M, Yang S, Xing X. Assessing prompt injection risks in 200+ custom GPTs. arXiv:2311.11538. 2023.
46. Zhang C, Jin M, Yu Q, Liu C, Xue H, Jin X. Goal-guided generative prompt injection attack on large language models. In: 2024 IEEE International Conference on Data Mining (ICDM). Piscataway, NJ, USA: IEEE; 2024. p. 941–6.
47. Zhang W, Kong X, Dewitt C, Braunl T, Hong JB. A study on prompt injection attack against LLM-integrated mobile robotic systems. In: 2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW). Piscataway, NJ, USA: IEEE; 2024. p. 361–8.
48. Alizadeh M, Samei Z, Stetsenko D, Gilardi F. Simple prompt injection attacks can leak personal data observed by LLM agents during task execution. arXiv:2506.01055. 2025.
49. Liang Z, Hu H, Ye Q, Xiao Y, Li H. Why are my prompts leaked? unraveling prompt extraction threats in customized large language models. arXiv:2408.02416. 2024.
50. Wang X, Bloch J, Shao Z, Hu Y, Zhou S, Gong NZ. WebInject: prompt injection attack to web agents. In: Christodoulopoulos C, Chakraborty T, Rose C, Peng V, editors. Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Stroudsburg, PA, USA: ACL; 2025. p. 2010–30.
51. Pearce H, Ahmad B, Tan B, Dolan-Gavitt B, Karri R. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Communicat ACM*. 2025;68(2):96–105. doi:10.1145/3610721.
52. Qu Y, Huang S, Li Y, Bai T, Chen X, Wang X, et al. BadCodePrompt: backdoor attacks against prompt engineering of large language models for code generation. *Autom Softw Eng*. 2025;32(1):17. doi:10.1007/s10515-024-00485-2.
53. Lian Z, Wang W, Zeng Q, Nakanishi T, Kitasuka T, Su C. Prompt-in-content attacks: exploiting uploaded inputs to hijack LLM behavior. arXiv:2508.19287. 2025.
54. Ye R, Pang X, Chai J, Chen J, Yin Z, Xiang Z, et al. Are we there yet? revealing the risks of utilizing large language models in scholarly peer review. arXiv:2412.01708. 2024.
55. Liu Y, Deng G, Xu Z, Li Y, Zheng Y, Zhang Y, et al. Jailbreaking chatgpt via prompt engineering: an empirical study. arXiv:2305.13860. 2023.
56. Zou A, Wang Z, Carlini N, Nasr M, Kolter JZ, Fredrikson M. Universal and transferable adversarial attacks on aligned language models. arXiv:2307.15043. 2023.
57. Edwards B. AI-powered Bing Chat spills its secrets via prompt injection attack [Internet]; 2023 [cited 2025 Nov 9]. Available from: <https://arstechnica.com/information-technology/2023/02/ai-powered-bing-chat-spills-its-secrets-via-prompt-injection-attack/>.
58. Willison S. Prompt Injection attacks against GPT-3 [Internet]; 2022 [cited 2025 Nov 9]. Available from: <https://simonwillison.net/2022/Sep/12/prompt-injection/>.
59. Kravitz D. How to steal intellectual property from GPTs [Internet]; 2024 [cited 2025 Nov 9]. Available from: <https://www.catonetworks.com/blog/how-to-steal-intellectual-property-from-gpts/>.
60. Monsell S. Task switching. *Trends Cogn Sci*. 2003;7(3):134–40.
61. Wallach W, Allen C. Moral machines: teaching robots right from wrong. Oxford, UK: Oxford University Press; 2008.

62. Pascanu R, Mikolov T, Bengio Y. On the difficulty of training recurrent neural networks. In: International Conference on Machine Learning. London, UK: PMLR; 2013. p. 1310–8.
63. Ioffe S, Szegedy C. Batch normalization: accelerating deep network training by reducing internal covariate shift. In: International Conference on Machine Learning. London, UK: PMLR; 2015. p. 448–56.
64. Bolukbasi T, Chang KW, Zou JY, Saligrama V, Kalai AT. Man is to computer programmer as woman is to homemaker? debiasing word embeddings. In: Advances in neural information processing systems. Red Hook, NY, USA: Curran Associates, Inc.; 2016.
65. Ba JL, Kiros JR, Hinton GE. Layer normalization. arXiv:1607.06450. 2016.
66. He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. Piscataway, NJ, USA: IEEE; 2016. p. 770–8.
67. Kirkpatrick J, Pascanu R, Rabinowitz N, Veness J, Desjardins G, Rusu AA, et al. Overcoming catastrophic forgetting in neural networks. Proc Nat Acad Sci U S A. 2017;114(13):3521–6. doi:10.1073/pnas.1611835114.
68. Madry A, Makelov A, Schmidt L, Tsipras D, Vladu A. Towards deep learning models resistant to adversarial attacks. arXiv:1706.06083. 2018.
69. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. In: Advances in neural information processing systems. Red Hook, NY, USA: Curran Associates, Inc; 2017.
70. Mimno D, Thompson L. The strange geometry of skip-gram with negative sampling. In: Empirical methods in natural language processing. Stroudsburg, PA, USA: ACL; 2017. p. 2873–8.
71. Adi Y, Kermany E, Belinkov Y, Lavi O, Goldberg Y. Fine-grained analysis of sentence embeddings using auxiliary prediction tasks. arXiv:1608.04207. 2016.
72. Smith SL, Kindermans PJ, Ying C, Le QV. Don't decay the learning rate, increase the batch size. arXiv:1711.00489. 2017.
73. Shaw P, Uszkoreit J, Vaswani A. Self-attention with relative position representations. In: Walker M, Ji H, Stent A, editors. Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Stroudsburg, PA, USA: ACL; 2018. p. 464–8.
74. Khandelwal U, He H, Qi P, Jurafsky D. Sharp nearby, fuzzy far away: how neural language models use context. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics; 2018 Jul 15–20; Melbourne, VIC, Australia. Stroudsburg, PA, USA: ACL; 2018. p. 284–94.
75. Fan A, Lewis M, Dauphin Y. Hierarchical neural story generation. In: Gurevych I, Miyao Y, editors. Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2018. p. 889–98.
76. Mauer M. The book of why-the new science of cause and effect. Rechtsatheorie. 2019;50:131.
77. Li H, Xu Z, Taylor G, Studer C, Goldstein T. Visualizing the loss landscape of neural nets. In: Advances in neural information processing systems. Red Hook, NY, USA: Curran Associates, Inc; 2018. p. 31.
78. Clark K, Khandelwal U, Levy O, Manning CD. What does BERT look at? an analysis of BERT's attention. In: Linzen T, Chrupała G, Belinkov Y, Hupkes D, editors. Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP. Stroudsburg, PA, USA: ACL; 2019. p. 276–86. doi:10.1017/s135132491900024x.
79. Michel P, Levy O, Neubig G. Are sixteen heads really better than one? In: Advances in neural information processing systems. Red Hook, NY, USA: Curran Associates, Inc.; 2019. 32 p.
80. Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I, et al. Language models are unsupervised multitask learners. OpenAI Blog. 2019;1(8):9.
81. Holtzman A, Buys J, Du L, Forbes M, Choi Y. The curious case of neural text degeneration. arXiv:1904.09751. 2019.
82. McCoy RT, Pavlick E, Linzen T. Right for the wrong reasons: diagnosing syntactic heuristics in natural language inference. In: Korhonen A, Traum D, Màrquez L, editors. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2019. p. 3428–48.

83. Wallace E, Feng S, Kandpal N, Gardner M, Singh S. Universal adversarial triggers for attacking and analyzing NLP. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing; 2019 Nov 3–7. Hong Kong, China. Stroudsburg, PA, USA: ACL; 2019. p. 2153–62.
84. Russell S. Human-compatible artificial intelligence. *Human-like Mach Intell.* 2022;1:3–22.
85. Kurita K, Vyas N, Pareek A, Black AW, Tsvetkov Y. Measuring bias in contextualized word representations. In: Costa-jussà MR, Hardmeier C, Radford W, Webster K, editors. Proceedings of the First Workshop on Gender Bias in Natural Language Processing. Stroudsburg, PA, USA: ACL; 2019. p. 166–72.
86. Tenney I, Das D, Pavlick E. BERT rediscovers the classical NLP pipeline. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics; 2019 Jul 28–Aug 2; Florence, Italy. Stroudsburg, PA, USA: ACL; 2019. p. 4593–601.
87. Voita E, Talbot D, Moiseev F, Sennrich R, Titov I. Analyzing multi-head self-attention: specialized heads do the heavy lifting, the rest can be pruned. In: Korhonen A, Traum D, Màrquez L, editors. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2019. p. 5797–808.
88. Hendrycks D, Carlini N, Schulman J, Steinhardt J. Unsolved problems in ML safety. arXiv:2109.13916. 2021.
89. Emma S, Ananya G, Andrew M. Energy and policy considerations for modern deep learning research. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 34; Palo Alto, CA, USA: AAAI Press; 2020. p. 13693–6.
90. Gabriel I. Artificial intelligence, values, and alignment. *Minds Mach.* 2020;30(3):411–37. doi:10.1007/s11023-020-09539-2.
91. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, et al. Language models are few-shot learners. *Adv Neural Inform Process Syst.* 2020;33:1877–901.
92. Beltagy I, Peters ME, Cohan A. Longformer: the long-document transformer. arXiv:2004.05150. 2020.
93. Glorot X, Bengio Y. Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics. JMLR Workshop and Conference Proceedings; 2010; Cambridge, MA, USA. p. 249–56.
94. Gehman S, Gururangan S, Sap M, Choi Y, Smith NA. RealToxicityPrompts: evaluating neural toxic degeneration in language models. In: Cohn T, He Y, Liu Y, editors. Findings of the Association for Computational Linguistics: EMNLP 2020. Stroudsburg, PA, USA: ACL; 2020. p. 3356–69.
95. Dathathri S, Madotto A, Lan J, Hung J, Frank E, Molino P, et al. Plug and play language models: a simple approach to controlled text generation. arXiv:1912.02164. 2019.
96. Xu J, Ju D, Li M, Boureau YL, Weston J, Dinan E. Recipes for safety in open-domain chatbots. arXiv:2010.07079. 2020.
97. Rogers A, Kovaleva O, Rumshisky A. A primer in BERTology: what we know about how BERT works. *Transact Associat Computat Linguist.* 2020;8:842–66. doi:10.1162/tacl_a_00349.
98. Dodge J, Ilharco G, Schwartz R, Farhadi A, Hajishirzi H, Smith N. Fine-tuning pretrained language models: weight initializations, data orders, and early stopping. arXiv:2002.06305. 2020.
99. Kenton Z, Everitt T, Weidinger L, Gabriel I, Mikulik V, Irving G. Alignment of language agents. arXiv:2103.14659. 2021.
100. Bender EM, Gebru T, McMillan-Major A, Shmitchell S. On the dangers of stochastic parrots: can language models be too big?. In: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency. New York, NY, USA: ACM; 2021. p. 610–23.
101. Press O, Smith NA, Lewis M. Train short, test long: attention with linear biases enables input length extrapolation. arXiv:2108.12409. 2021.
102. Bai Y, Kadavath S, Kundu S, Askell A, Kernion J, Jones A, et al. Constitutional ai: harmlessness from AI feedback. arXiv:2212.08073. 2022.
103. Lee K, Ippolito D, Nystrom A, Zhang C, Eck D, Callison-Burch C, et al. Deduplicating training data makes language models better. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2022. p. 8424–45.

104. Gao L, Schulman J, Hilton J. Scaling laws for reward model overoptimization. In: International Conference on Machine Learning. London, UK: PMLR; 2023. p. 10835–66.
105. Li B. Ethical issues for literary translation in the Era of artificial intelligence. *Babel*. 2023;69(4):529–45.
106. Yong ZX, Menghini C, Bach SH. Low-resource languages jailbreak GPT-4. arXiv:2310.02446. 2023.
107. Deng Y, Zhang W, Pan SJ, Bing L. Multilingual jailbreak challenges in large language models. arXiv:2310.06474. 2023.
108. Dai Z, Yang Z, Yang Y, Carbonell JG, Le Q, Salakhutdinov R. Transformer-xl: attentive language models beyond a fixed-length context. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2019. p. 2978–88.
109. Lazaridou A, Kuncoro A, Gribovskaya E, Agrawal D, Liska A, Terzi T, et al. Mind the gap: assessing temporal generalization in neural language models. *Adv Neural Inform Process Syst*. 2021;34:29348–63.
110. Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, et al. Training language models to follow instructions with human feedback. *Adv Neural Inform Process Syst*. 2022;35:27730–44.
111. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *J Mach Learn Res*. 2020;21(140):1–67.
112. Amodei D, Olah C, Steinhardt J, Christiano P, Schulman J, Mané D. Concrete problems in AI safety. arXiv:1606.06565. 2016.
113. Elazar Y, Kassner N, Ravfogel S, Ravichander A, Hovy E, Schütze H, et al. Measuring and improving consistency in pretrained language models. *Transact Associat Computat Linguist*. 2021;9(1):1012–31. doi:10.1162/tac1_a_00410.
114. Gu A, Mamba Dao T. Linear-time sequence modeling with selective state spaces. arXiv:2312.00752. 2024.
115. Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, et al. Gpt-4 technical report. arXiv:2303.08774. 2023.
116. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv Neural Inform Process Syst*. 2020;33:9459–74.
117. Kumar A, Agarwal C, Srinivas S, Li AJ, Feizi S, Lakkaraju H. Certifying LLM safety against adversarial prompting. arXiv:2309.02705. 2024.
118. Madaan A, Tandon N, Gupta P, Hallinan S, Gao L, Wiegrefe S, et al. Self-refine: iterative refinement with self-feedback. *Adv Neural Inform Process Syst*. 2023;36:46534–94.
119. Rebedea T, Dinu R, Sreedhar MN, Parisien C, Cohen J. Nemo guardrails: a toolkit for controllable and safe LLM applications with programmable rails. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Stroudsburg, PA, USA: ACL; 2023. p. 431–45.
120. Choi E, Jo Y, Jang J, Seo M. Prompt injection: parameterization of fixed inputs. arXiv:2206.11349. 2022.
121. Zhang Z, Yang J, Ke P, Mi F, Wang H, Huang M. Defending large language models against jailbreaking attacks through goal prioritization. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2024. p. 8865–87.
122. Chen Y, Li H, Zheng Z, Wu D, Song Y, Hooi B. Defense against prompt injection attack by leveraging attack techniques. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2025. p. 18331–47.
123. Hines K, Lopez G, Hall M, Zarfati F, Zunger Y, Kiciman E. Defending against indirect prompt injection attacks with spotlighting. arXiv:2403.14720. 2024.
124. Khomsky D, Maloyan N, Nutfullin B. Prompt injection attacks in defended systems. In: International Conference on Distributed Computer and Communication Networks. Cham, Switzerland: Springer; 2024. p. 404–16.
125. Phute M, Helbling A, Hull M, Peng S, Szyller S, Cornelius C, et al. Llm self defense: by self examination, LLMS know they are being tricked. arXiv:2308.07308. 2023.
126. Rai P, Sood S, Madiseti VK, Bahga A. Guardian: a multi-tiered defense architecture for thwarting prompt injection attacks on llms. *J Softw Eng Applicat*. 2024;17(1):43–68. doi:10.4236/jsea.2024.171003.
127. Wang J, Wu F, Li W, Pan J, Suh E, Mao ZM, et al. Fath: authentication-based test-time defense against indirect prompt injection attacks. arXiv:2410.21492. 2024.

128. Chen S, Piet J, Sitawarin C, Wagner D. {StruQ}: defending against prompt injection with structured queries. In: 34th USENIX Security Symposium (USENIX Security 25). Berkeley, CA, USA: USENIX Association; 2025. p. 2383–400.
129. Jia F, Wu T, Qin X, Squicciarini A. The task shield: enforcing task alignment to defend against indirect prompt injection in LLM agents. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2025. p. 29680–97.
130. Pasquini D, Kornaropoulos EM, Ateniese G. Hacking back the AI-hacker: prompt injection as a defense against LLM-driven cyberattacks. arXiv:2410.20911. 2024.
131. Suo X. Signed-prompt: a new approach to prevent prompt injection attacks against LLM-integrated applications. AIP Conf Proc. 2024;3194:040013. doi:10.1063/5.0222987.
132. Wu F, Cecchetti E, Xiao C. System-level defense against indirect prompt injection attacks: an information flow control perspective. arXiv:2409.19091. 2024.
133. Chen S, Zharmagambetov A, Mahlouljifar S, Chaudhuri K, Wagner D, Guo C. SecAlign: defending against prompt injection with preference optimization. In: CCS '25: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security; 2025. New York, NY, USA: ACM. p. 2833–47.
134. Jacob D, Alzahrani H, Hu Z, Alomair B, Wagner D. Promptshield: deployable detection for prompt injection attacks. In: Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy. New York, NY, USA: ACM; 2024. p. 341–52.
135. Panterino S, Fellington M. Dynamic moving target defense for mitigating targeted LLM prompt injection. Authorea Preprints. 2024. doi: 10.36227/techrxiv.171822345.56781952/v1.
136. Piet J, Alrashed M, Sitawarin C, Chen S, Wei Z, Sun E, et al. Jatmo: prompt injection defense by task-specific finetuning. In: European Symposium on Research in Computer Security. Cham, Switzerland: Springer; 2024. p. 105–24.
137. Chen Y, Li H, Sui Y, Liu Y, He Y, Song Y, et al. Robustness via referencing: defending against prompt injection attacks by referencing the executed instruction. arXiv:2504.20472. 2025.
138. Lin H, Lao Y, Geng T, Yu T, Zhao W. Uniguardian: a unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models. arXiv:2502.13141. 2025.
139. Shi T, Zhu K, Wang Z, Jia Y, Cai W, Liang W, et al. Promptarmor: simple yet effective prompt injection defenses. arXiv:2507.15219. 2025.
140. Yi J, Xie Y, Zhu B, Kiciman E, Sun G, Xie X, et al. Benchmarking and defending against indirect prompt injection attacks on large language models. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining. New York, NY, USA: ACM; 2025. p. 1809–20.
141. Zhong PY, Chen S, Wang R, McCall M, Titzer BL, Miller H, et al. Rtbas: defending LLM agents against prompt injection and privacy leakage. arXiv:2502.08966. 2025.
142. Li H, Liu X. Injecguard: benchmarking and mitigating over-defense in prompt injection guardrail models. arXiv:2410.22770. 2024.
143. Li H, Liu X, Zhang N, Xiao C. PIGuard: prompt injection guardrail via mitigating overdefense for free. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2025. p. 30420–37.
144. Liu Y, Jia Y, Jia J, Song D, Gong NZ. DataSentinel: a game-theoretic detection of prompt injection attacks. In: 2025 IEEE Symposium on Security and Privacy (SP). Piscataway, NJ, USA: IEEE; 2025. p. 2190–208.
145. Zhang R, Sullivan D, Jackson K, Xie P, Chen M. Defense against prompt injection attacks via mixture of encodings. In: Chiruzzo L, Ritter A, Wang L, editors. Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies. Stroudsburg, PA, USA: ACL; 2025. p. 244–52.
146. Sharma RK, Gupta V, Grossman D. Defending language models against image-based prompt attacks via user-provided specifications. In: 2024 IEEE Security and Privacy Workshops (SPW). Piscataway, NJ, USA: IEEE; 2024. p. 112–31. doi: 10.1109/spw63631.2024.00017.

147. Wen T, Wang C, Yang X, Tang H, Xie Y, Lyu L, et al. Defending against indirect prompt injection by instruction detection. In: Christodoulopoulos C, Chakraborty T, Rose C, Peng V, editors. Findings of the Association for Computational Linguistics: EMNLP 2025. Stroudsburg, PA, USA: ACL; 2025. p. 19472–87.
148. Salem A, Paverd A, Köpf B. Maatphor: automated variant analysis for prompt injection attacks. arXiv:2312.11513. 2023.
149. Hung KH, Ko CY, Rawat A, Chung IH, Hsu WH, Chen PY. Attention tracker: detecting prompt injection attacks in LLMs. In: Findings of the Association for Computational Linguistics: NAACL 2025. Stroudsburg, PA, USA: ACL; 2025. p. 2309–22.
150. Ayub MA, Majumdar S. Embedding-based classifiers can detect prompt injection attacks. In: Proceedings of the CAMLIS'24: Conference on Applied Machine Learning for Information Security; 2024 Oct 24–25; Arlington, VA, USA.
151. Rahman MA, Shahriar H, Wu F, Cuzzocrea A. Applying pre-trained multilingual bert in embeddings for improved malicious prompt injection attacks detection. In: 2024 2nd International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings). Piscataway, NJ, USA: IEEE; 2024. p. 1–7.
152. Chen Y, Li H, Sui Y, He Y, Liu Y, Song Y, et al. Can indirect prompt injection attacks be detected and removed?. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA, USA: ACL; 2025. p. 18189–206.
153. Li R, Chen M, Hu C, Chen H, Xing W, Han M. Gentel-safe: a unified benchmark and shielding framework for defending against prompt injection attacks. arXiv:2409.19521. 2024.
154. Sharma RK, Gupta V, Grossman D. SPML: a DSL for defending language models against prompt attacks. arXiv:2402.11755. 2024.
155. Pedro R, Castro D, Carreira P, Santos N. From prompt injections to SQL injection attacks: how protected is your LLM-integrated web application? arXiv:2308.01990. 2023.
156. Lee Y, Park T, Lee Y, Gong J, Kang J. Exploring potential prompt injection attacks in federated military LLMs and their mitigation. arXiv:2501.18416. 2025.
157. Zhan Q, Fang R, Panchal HS, Kang D. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In: Chiruzzo L, Ritter A, Wang L, editors. Findings of the Association for Computational Linguistics: NAACL 2025. Stroudsburg, PA, USA: ACL; 2025. p. 7101–17.
158. Yu J, Shao Y, Miao H, Shi J. Promptfuzz: harnessing fuzzing techniques for robust testing of prompt injection in LLMs. arXiv:2409.14729. 2024.
159. Deng B, Wang W, Feng F, Deng Y, Wang Q, He X. Attack prompt generation for red teaming and defending large language models. In: Bouamor H, Pino J, Bali K, editors. Findings of the Association for Computational Linguistics: EMNLP 2023. Stroudsburg, PA, USA: ACL; 2023. p. 2176–89.
160. Mudarova R, Namiot D. Countering Prompt Injection attacks on large language models. Int J Open Inform Technol. 2024;12(5):39–48.
161. Li Z, Peng B, He P, Yan X. Evaluating the instruction-following robustness of large language models to prompt injection. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Stroudsburg, PA, USA: ACL; 2024. p. 557–68.
162. Yip DW, Esmradi A, Chan CF. A novel evaluation framework for assessing resilience against prompt injection attacks in large language models. In: 2023 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE). Piscataway, NJ, USA: IEEE; 2023. p. 1–5.
163. Sang X, Gu M, Chi H. Evaluating prompt injection safety in large language models using the promptbench dataset. Open Science Framework. 2024. doi: 10.31219/osf.io/7zck8.
164. Ramakrishna A, Majumdar J, Gupta R, Hazarika D. LLM-PIRATE: a benchmark for indirect prompt injection attacks in large language models. In: AdvML-Frontiers'24: The 3rd Workshop on New Frontiers in Adversarial Machine Learning@NeurIPS'24, 2024 Dec 12–14; Vancouver, BC, Canada. p. 1–10.
165. Evtimov I, Zharmagambetov A, Grattafiori A, Guo C, Chaudhuri K. Wasp: benchmarking web agent security against prompt injection attacks. arXiv:2504.18575. 2025.
166. Chao P, DeBenedetti E, Robey A, Andriushchenko M, Croce F, Sehwag V, et al. Jailbreakbench: an open robustness benchmark for jailbreaking large language models. Adv Neural Inform Process Syst. 2024;37:55005–29.